

Missing Value Imputation using Denoising Diffusion GAN with Modular Plugins

(Technical Report)

Shuang Hao Chang Liu Xiang Huang

School of Computer Science and Technology, Beijing Jiaotong University, China
Beijing Key Laboratory of Traffic Data Mining and Embodied Intelligence, China
{haoshuang, 25120424, 22120382}@bjtu.edu.cn

Abstract—Dealing with missing values in relational tables is a pervasive challenging preparation phase before proceeding to data analysis and mining. Directly removing missing data often causes the loss of information and thus adversely impacts further data processing. Missing values can also be estimated using extensively studied imputation techniques. However, they seldom consider improving the utility of imputed data for the downstream task, and rarely try to maintain the inherent data dependencies. In this paper, we introduce DDGANI, a novel missing value imputation framework based on denoising diffusion GAN. Central to our approach is the utilization of denoising diffusion GAN for effective data imputation. Additionally, we incorporate a self-attention mechanism, enabling the model to focus on more relevant information from similar tuples. Our framework also integrates two plugins that can be chosen based on data characteristics and downstream tasks, which ensure the utility of imputed data and the preservation of inherent data dependencies. Extensive experiments on multiple datasets demonstrate the superiority of DDGANI compared to state-of-the-art imputation techniques and verify the effectiveness of two modular plugins.

Index Terms—Missing value, Denoising diffusion GAN, Self-attention mechanism, Data utility, Data dependencies

I. INTRODUCTION

Data are undoubtedly an essential element of decision-making, so high-quality data is a necessity. However, in real-world scenarios, data often go missing, which may arise due to human error during data collection or equipment malfunctions. Although some factors leading to missing values are perceived, they are unavoidable. Therefore, it is essential to address missing data before diving into data analysis and mining.

Existing Approaches and Limitations. Manually imputing these missing values is undoubtedly a time-consuming and labor-intensive task. A straightforward approach might be to remove these missing data. However, neglecting such missing values can distort the analysis or lead to inaccuracies, ultimately affecting the downstream decision-making process [2]. Thus, the most common strategy for addressing missing values is to reconstruct a complete dataset using data imputation techniques. Most of the earlier studies focus on the basic statistical imputation methods, utilizing statistics such as the mean, median, and mode to impute the missing values. However, most of these methods work only well for numerical data. Traditional machine learning (ML) imputation

solutions train an ML model to predict missing cells. The accuracy of imputation is limited by the predictive power of the ML model. In addition, some of them have strict distributional assumptions, such as PCAI [19], and some are less effective for categorical data, such as KNN [1]. Heuristic-based methods utilize data imputation rules to deal with missing values, such as relaxed functional dependencies [6] and differential dependencies [34]. However, they mainly focus on repairing categorical values (*e.g.*, City, State), and the design of high-quality imputation rules requires extensive domain knowledge [23]. Moreover, they sometimes fail to provide a deterministic imputation solution for attributes on the left-hand side of these data dependencies. For example, for the missing value $t_5[\text{AreaCode}]$ in Table I, these methods may give two possible imputations, since both “501” and “870” satisfy the given imputation rules.

Recently, deep learning (DL) based approaches, mainly consisting of methods based on deep generative models and pre-trained language models, have provided an effective way to tackle the above obstacles and have obtained impressive results on the missing value imputation problem. Generative adversarial networks (GAN) and variational autoencoder (VAE) are two common deep generative models adopted in previous work [13], [25], [28], [43]. However, they have difficulty directly generating samples from a complex distribution in one shot and are confined to Missing Completely At Random (MCAR) settings [17]. Recent work has attempted to utilize diffusion models for missing value imputation [33], [40]. But they often neglect to check whether the imputed table violates the original data dependencies. For example, given the similar distribution between t_4 and t_5 in Table I, most of these methods would impute $t_4[\text{State}]$ as “AR” resulting in a violation of φ_1 . Actually, “Neward” in t_4 and t_5 are different cities in different states despite having the same name. With the prominence of pre-trained language models (PLMs) in multiple data processing tasks, there are PLM-based imputation methods such as RPT [36]. These PLM-based imputation methods do not handle numerical data well (*e.g.*, Rate). Furthermore, the vast majority of the above often fail to ensure the applicability of the imputed data for subsequent tasks, which is a significant challenge at present [27].

TABLE I: Example of a Relational Table with Missing Values

Tuple ID	Fname	AreaCode	City	State	MaritalStatus	SingleExemp	Salary	Rate	HasChild (label)
t_1	Muneo	201	Hampton	NJ	M	0	10000	1.40	Yes
t_2	Hyujin	201	Hampton	NJ	M	0	[90000]	2.50	Yes
t_3	Jarno	201	Hampton	NJ	S	[1000]	80000	2.34	No
t_4	Coke	201	Newark	[NJ]	M	0	5000	[1.40]	No
t_5	Ada	[501]	Newark	AR	M	0	5000	1.38	Yes
t_6	Liesbeth	501	Newark	AR	S	20	[5000]	1.38	No
t_7	Refik	501	[Newark]	AR	S	20	10000	2.58	Yes
t_8	Sreerupa	870	Sage	AR	[S]	20	[5000]	1.38	No

[†] A gray cell means it is missing and an **turquoise** colored word indicates the true value

[‡] The FDs that hold in this table are $\varphi_1 : \text{AreaCode} \rightarrow \text{State}$ and $\varphi_2 : \text{AreaCode}, \text{MaritalStatus} \rightarrow \text{SingleExemp}$

Our Proposal. To address the aforementioned limitations, we propose a novel missing value imputation framework DDGANI as depicted in Figure 1. It contains a data imputation module, which is the mainstay of the framework, and two optional plug-ins: a data utility plug-in and a data dependency plug-in. We employ the denoising diffusion GAN [42] in the data imputation module to reconstruct the complete dataset, which takes the relational table with missing values as input and outputs the imputed table. This choice is motivated by its ability to model complex data distributions through several conditional denoising diffusion steps, allowing for the proficient generation of various types of data and achieving high data quality within an acceptable timeframe. In addition, we incorporate the self-attention mechanism, which enables the generative model to capture more relevant contextual information when dealing with missing data.

We introduce a data utility plug-in to ensure the utility of the imputed data for the subsequent classification task via a meta-learning paradigm. We fully use the downstream classifier and validation set, with the goal that the classifier trained with imputed data could minimize the loss on the validation set. It is achieved by backpropagating the validation loss to the data generation module to guarantee the utility of the imputed table. If the downstream classifier or validation set is unavailable, the data utility plug-in builds a two-layer fully connected neural network acting as the downstream classifier and selects some tuples from the incomplete table where all attributes are observed to serve as a validation set. This can also contribute to keeping the same data distribution between the imputed and observed data.

We also design a data dependency plug-in to allow the imputed data to adhere to the original data dependencies among the categorical attributes. In this paper, we describe how it works using function dependencies (FDs) as an example. This plug-in could also support other constraints such as conditional functional dependencies (CFDs). This module initially mines potential FDs from the table with missing values. Each FD $\varphi : Z \rightarrow A$ is then modeled by a classifier, which takes the values of attributes Z on the left-hand side of the FD as input and predicts the attribute value of A (these models could also be pre-trained using the master data or knowledge bases). We minimize the difference between the predicted values given by the dependency models and the imputed values given by the data imputation module to prevent the imputed data from violating existing data constraints. For

example, given $\varphi_1 : \text{AreaCode} \rightarrow \text{State}$, imputation of $t_4[\text{State}]$ in Table I with “AR” would result in a penalty in the loss of the generative model. Consequently, DDGANI would lean towards imputing it with “NJ”. However, the automatically generated FDs from an incomplete dataset may not be fully trusted. As the imputation becomes more accurate, we must go back to determine whether these FDs are valid based on the confidence of constraints and violated tuples. Intuitively, if the violated tuples are more trusted, we refine the FDs and their corresponding dependency models.

Each component in our framework, though appearing to have its own distinct goal, works together to minimize imputation error. The attention module and two plug-ins further improve the ability of the base model to capture the correlations between attributes and between attributes and the label in different ways. Meanwhile, this modular design can make the entire framework more flexible for different data characteristics and downstream tasks.

Contributions. Our contributions are summarized as follows:

- We propose a novel missing value imputation framework with a self-attention-enhanced generative model for tabular data imputation and two plug-ins that can be chosen to improve performance based on data characteristics and downstream tasks.
- We adapt the denoising diffusion GAN for missing value imputation (Section III), which could achieve high-quality imputation results with enhanced efficiency. **To overcome its weakness in capturing long-range dependencies within tabular data, we introduce a model-agnostic table-specific attention mechanism that jointly captures inter-attribute and inter-tuple correlations.**
- We design a data utility plug-in to boost the performance of our imputation framework via a meta-learning paradigm, which guarantees the applicability of our imputed data for training downstream models for a target task (Section IV).
- **We propose a data dependency plug-in to ensure the preservation of inherent data dependencies in the imputed relational table, which integrates FD mining from incomplete data, FD modeling, and iterative refinement based on the confidence of mined FDs and imputed tuples.** (Section V).
- We conducted extensive experiments on multiple datasets with categorical and numerical attributes. The experimental results demonstrate that our approach outperforms a variety of state-of-the-art missing value imputation methods. (Section VII).

II. PROBLEM FORMULATION

Data Model. Let X denote a relational table with n tuples $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and k attributes $U = \{A_1, \dots, A_k\}$. Each attribute is either numerical or categorical. We use x_{ij} to denote the value of attribute A_j in tuple $\mathbf{x}_i$. Each tuple $\mathbf{x}_i$ has a label y_i for classification. Note that for ease of illustration, we sometimes refer to each tuple by its ID t_i in Table I and use $t_i[A_j]$ to denote its value of the attribute A_j .

Missing Data. Each tuple $\mathbf{x}$ may have some attribute values that are not observed. To formalize these missing values, we follow the existing work [21] to introduce a *mask* matrix $\mathbf{M} = \{\mathbf{m}_1, \dots, \mathbf{m}_n\}$ to indicate which value in X is observed. Each $\mathbf{m}_i = \{m_{i1}, \dots, m_{ik}\}$ represents the missing status of the tuple $\mathbf{x}_i$: x_{ij} is observed if $m_{ij} = 1$, otherwise x_{ij} is missing.

Data Dependency. A dependency is a constraint that defines the relationship between attributes. Let us take the functional dependency (FD) as an example. We say X satisfies the FD $\varphi : Z \rightarrow A$ (Z, A are two disjoint subsets of attributes, and the right-hand side A of φ has only one attribute) if, for any tuple, the attribute values in Z uniquely determine the attribute value in A .

Missing Data Imputation. The missing data imputation problem is to replace the missing data in X with an estimated value based on other observed values. If $\bar{X} = \{\bar{x}_1, \dots, \bar{x}_n\}$ denotes the imputed table and $X^\vee = \{x_1^\vee, \dots, x_n^\vee\}$ denotes the complete ground-truth table, which is not available in practical situations, our goal is to minimize the difference between $\bar{X}$ and $X^\vee$.

III. MISSING VALUE IMPUTATION NETWORK

A. Denoising Diffusion GAN

Denoising Diffusion Probabilistic Model. The denoising diffusion probabilistic model defines a forward diffusion process that adds varying degrees of Gaussian noise to the original data $X_0 \sim q(X_0)$ over T steps where $q(X_0)$ is the original data distribution. The forward process at step t can be represented as $q(X_t|X_{t-1})$ where X_t denotes the data obtained by adding noise to X_{t-1} , i.e., to X_0 for the t steps. The reverse diffusion process is to guess the Gaussian noise added in the forward process and remove iteratively (within T steps), using a neural network model Φ_θ . The reverse process can be represented as $p_\theta(X_{t-1}|X_t)$ where θ represents the parameters of the model. However, due to the typically unknown nature of $q(X_{t-1}|X_t)$ in the diffusion model, it is assumed that the size of the denoising step T is on the order of thousands of steps, and the diffusion rate β_t in each step should be small, making its training time-consuming.

Denoising Diffusion GAN. To address the above issue, Xiao et al. [42] employed conditional GANs to approximate $q(X_{t-1}|X_t)$. That is, instead of maintaining a very small β_t to approximate $q(X_{t-1}|X_t)$ with $q(X_t|X_{t-1})$, it can directly utilize conditional GANs to learn $q(X_{t-1}|X_t)$. Therefore, a large diffusion rate β_t and a small denoising diffusion step

T could be adopted. That is why we choose the denoising diffusion GAN for missing value imputation.

B. Denoising Diffusion GAN For Data Imputation

Attention Mechanisms. We first preprocess X with the self-attention mechanism, which encourages the subsequent generation model to pay more attention to the observed data associated with missing values.

We build an attention map $\mathcal{A}^j$ for each attribute A_j , of size $n \times n$ where n is the number of tuples. Then, considering a missing value x_{ij} , intuitively, tuples that are more similar to $\mathbf{x}_i$ on attribute A_j deserve more attention, thus we have

$$x'_{ij} = \sum_h s(\mathbf{x}_i, \mathbf{x}_h)_j \times x_{hj}. \quad (1)$$

Here, $s(\mathbf{x}_i, \mathbf{x}_h)_j$ is the value in row i and column h of $\mathcal{A}^j$, representing the similarity between x_{ij} and x_{hj} . Notice that x_{ij} is missing, so the question here is how to calculate the similarity to a missing value. In fact, we can estimate the similarity of two tuples in attribute A_j by comparing their similarity in other attributes, and the more correlated an attribute is with A_j , the more effective it is to estimate the similarity in attribute A_j .

The next question is how to calculate the correlation between two attributes. We build a symmetric correlation matrix $\mathbf{C}$ while each element c_{ij} represents the correlation between attribute A_i and A_j . For two attributes A_i and A_j , we set $c_{ij} = 0$ if $i = j$. Otherwise, we pick all tuples with no missing values in both A_i and A_j and quantify the strength of correlation between them (denoted by c_{ij} and also c_{ji}) by the absolute value of *Cramér's V coefficient* [9].

Then, given two tuples $\mathbf{x}_i$ and $\mathbf{x}_h$, the similarity between x_{ij} and x_{hj} is measured with the weighted sum of the similarity of these two tuples in other attributes and the weights are determined by the strength of correlation with A_j . That is,

$$s(\mathbf{x}_i, \mathbf{x}_h)_j = \sum_z c_{zj} \times \text{sim}(x_{iz}, x_{hz}). \quad (2)$$

The similarity $\text{sim}(x_{iz}, x_{hz})$ is defined as follows. If A_z is a categorical attribute, $\text{sim}(x_{iz}, x_{hz}) = 1$ when x_{iz} and x_{hz} have the same value, otherwise $\text{sim}(x_{iz}, x_{hz}) = 0$. If A_z is a numerical attribute, $\text{sim}(x_{iz}, x_{hz}) = 1 - \frac{|x_{iz} - x_{hz}|}{\max_z - \min_z}$ where $\max_z$ and $\min_z$ are the maximum and minimum values of attribute A_z respectively. Then, each row $\mathcal{A}_i^j$ of the attention map $\mathcal{A}^j$ will be

$$\mathcal{A}_i^j = \text{Softmax}(s(\mathbf{x}_i, \mathbf{x}_1)_j, \dots, s(\mathbf{x}_i, \mathbf{x}_{n-1})_j, s(\mathbf{x}_i, \mathbf{x}_n)_j). \quad (3)$$

Remark. (1) Equation 1 only applies to the case where x_{ij} is missing but x_{hj} is not. When x_{hj} is also missing, $s(\mathbf{x}_i, \mathbf{x}_h)_j = 0$. When x_{ij} is not missing, we will utilize $\mathbf{M}$ to mask out x'_{ij} as Equation 4 (there is no need to utilize the attention mechanisms for an observed value). (2) When traversing the other attributes A_z of tuples $\mathbf{x}_i$ and $\mathbf{x}_h$ in Equation 2, if x_{iz} or x_{hz} is missing, we skip their similarity computation on attribute A_z as the above example shows. (3) We utilize a

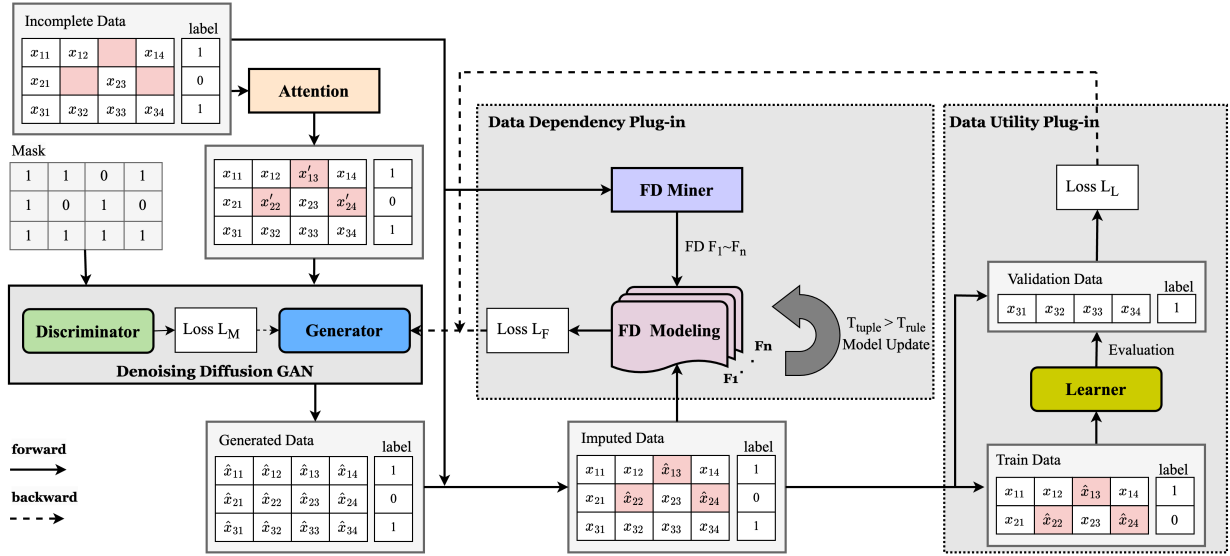


Fig. 1: The Architecture of DDGANI

threshold κ to set the values less than κ in $\mathcal{A}_i^j$ to 0, which could help to prevent the decision bias caused by data imbalance.

Let X_j be the projection of X on attribute A_j , and X^A is the concatenation of the product $\mathcal{A}^j \times X_j$ across all attributes. We can then reconstruct X by

$$X' = (\mathbf{1} - M) \odot X^A + M \odot X \quad (4)$$

where $\odot$ denotes element-wise multiplication and $\mathbf{1}$ is all-one matrix. We only use attention mechanisms during the data preprocessing phase and not after each iteration. This is because the cost of calculating similarities per iteration increases significantly with large datasets, but does not greatly enhance the final results.

Generator. After obtaining X' , we further impute the missing values with the use of the denoising diffusion GAN, which consists of a forward diffusion process and a posterior sampling process, as presented in Figure 2. To ease the following description, we denote X' by X'_0 , which is the output of the attention mechanism and the input of the following diffusion model.

In the forward diffusion process, we iteratively add Gaussian noise to X'_0 over T steps. In each step, we have:

$$q(X'_t | X'_{t-1}) = \mathcal{N}(X'_t; \sqrt{1 - \beta_t} X'_{t-1}, \beta_t \mathbf{I}). \quad (5)$$

Here, $\mathcal{N}$ represents the Gaussian distribution; t denotes any step between 1 and T ; β_t is the diffusion rate, representing the degree of noise added at step t ; X'_t denotes the data achieved by introducing noise into X'_0 over t steps; $\mathbf{I}$ denotes the identity matrix of the same dimension as X'_t .

In the posterior sampling process, we employ a generator G to perform denoising, *i.e.*, approximating $q(X'_{t-1} | X'_t)$. However, instead of taking X'_t as input and outputting X'_{t-1} , the generator will try to estimate X'_0 and then add noise to obtain X'_{t-1} . In this way, G has the singular goal of generating X'_0 , rather than a series of X'_{t-1} for different step t , which is beneficial for effective training of G .

Concretely, let $\hat{X}_0$ denote the actual output of G to be as similar as X'_0 in the observed data. We inject the noise into $\hat{X}_0$ again to get $\hat{X}_{t-1}$:

$$q(\hat{X}_{t-1} | X'_t, \hat{X}_0) = \mathcal{N}(\hat{X}_{t-1}; \tilde{\mu}_t(X'_t, \hat{X}_0), \tilde{\beta}_t \mathbf{I}) \quad (6)$$

where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, $\tilde{\mu}_t(X'_t, \hat{X}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \hat{X}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} X'_t$ and $\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$. And we define the imputed table $\bar{X}_{t-1}$ in step $t-1$ to be the combination of the partial observations from X'_{t-1} and imputed data from $\hat{X}_{t-1}$:

$$\bar{X}_{t-1} = (\mathbf{1} - M) \odot \hat{X}_{t-1} + M \odot X'_{t-1} \quad (7)$$

Subsequently, we use $\bar{X}_{t-1}$ as the input to G and obtain a new $\hat{X}_0$. Then we add noise to $\hat{X}_0$ to generate $\hat{X}_{t-2}$ by Equation 6, and derive $\bar{X}_{t-2}$ using Equation 7. This process continues until we generate the final imputed table $\bar{X}_0$ (*i.e.*, $\bar{X}$).

Discriminator. To make the fake (imputed) values in $\bar{X}$ to be as compatible as possible with the real (observed) values, some methods [14], [43] introduce a discriminator D to distinguish which components of $\bar{X}$ are real or fake. D serves as an adversary to the generator G , ensuring that the generated data closely resembles the real data. While these approaches can effectively enhance the performance of G , the potential pitfall is that D will most likely collapse to a few states since the real value it sees is fixed each time.

In order to make G better approximate $q(X'_{t-1} | X'_t)$ while increasing the sample diversity, in our framework, D takes X'_t , $\bar{X}_{t-1}$ and t as inputs, identify whether each attribute value $\bar{x}_{ij}$ comes from X'_{t-1} or $\hat{X}_{t-1}$, equivalently predicting M . In this way, D does not just consider individual positive samples but also their noisy versions, making it easier to grasp the data distribution and counteracting model overfitting.

Objective. Let's summarize the objective functions of the generator G and discriminator D . The output of G at each step is $\hat{X}_0$, to be as similar to X'_0 in terms of the observed

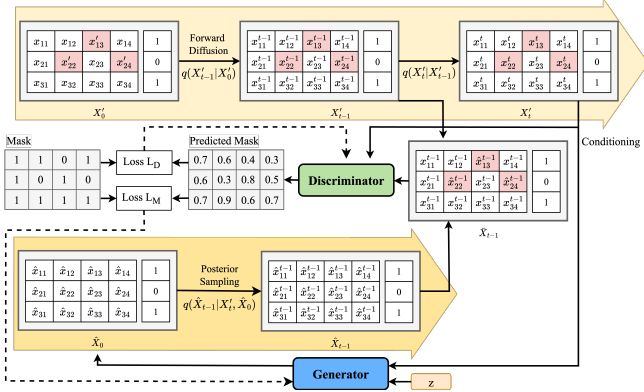


Fig. 2: The Training Process of Data Imputation

data. Therefore, G needs to minimize the reconstruction loss $\mathcal{L}_R$:

$$\mathcal{L}_R = \mathbf{M} \odot R(\hat{\mathbf{X}}_0, \mathbf{X}'_0) \quad (8)$$

where $R(\cdot)$ represents the calculation of mean squared error (MSE) loss for numerical data and cross-entropy loss for categorical data.

As adversarial training, G is meant to make D unable to distinguish which components of $\hat{\mathbf{X}}_{t-1}$ are real in each step, ensuring consistency of the distributions of $p_\theta(\mathbf{X}_{t-1}|\mathbf{X}_t)$ and $q(\mathbf{X}_{t-1}|\mathbf{X}_t)$. Therefore, G should minimize the loss $\mathcal{L}_M$:

$$\mathcal{L}_M = -\mathbb{E}_{\hat{\mathbf{X}}_t, t, \mathbf{M}}[(\mathbf{1} - \mathbf{M}) \log(D(\hat{\mathbf{X}}_t, t))]. \quad (9)$$

For $m_{ij} = 1$, the corresponding loss is 0. And for $m_{ij} = 0$, minimizing $\mathcal{L}_M$ is to incentivize the discriminator to predict it as 1, i.e., the quality of the imputed data is sufficient to pass off as real.

In each step t , we train D to maximize the probability of correctly predicting $\mathbf{M}$. Consequently, we define the loss $\mathcal{L}_D$ for the discriminator as:

$$\mathcal{L}_D = -\mathbb{E}_{\hat{\mathbf{X}}_t, t, \mathbf{M}}[\mathbf{M} \log D(\hat{\mathbf{X}}_t, t) + (\mathbf{1} - \mathbf{M}) \log(1 - D(\hat{\mathbf{X}}_t, t))]. \quad (10)$$

Minimizing $\mathcal{L}_D$ means minimizing the difference between the predicted mask $D(\hat{\mathbf{X}}_t, t)$ and the ground truth $\mathbf{M}$, which is predetermined by $\mathbf{X}$.

IV. DATA UTILITY PLUG-IN

If the imputed relational table $\hat{\mathbf{X}}$ is taken as training data, the data utility plug-in can be utilized to ensure the applicability of the imputed data for subsequent classification tasks, which has been introduced in our short paper [14]. We utilize a *Learner* L, which acts as the classifier used in the subsequent tasks. Thus, we expect the imputed data $\hat{\mathbf{X}}$ to minimize the learner's classification loss on the given validation set. This validation loss is denoted by $\mathcal{L}_L$ in Figure 1, which affects the update of the generator's parameters via backward propagation to generate the expected imputed data.

Suppose the downstream classifier and the validation data are available. We can directly employ them in this plug-in: training the downstream classifier while imputing the missing values in the training data. Otherwise, we construct the learner L using a two-layer fully connected neural network, and the

validation data will come from $\hat{\mathbf{X}}$. Specifically, we randomly take a portion of the tuples from which all its attribute values are observed in $\hat{\mathbf{X}}$ as the validation set $\mathbf{X}_v$, and the rest of the tuples in $\hat{\mathbf{X}}$ as the training set $\mathbf{X}_\eta$ to train L. The validation loss is determined by the difference between the labels obtained by applying L to the validation set and the true labels:

$$\mathcal{L}_L = \frac{1}{|\mathbf{X}_v|} \sum_{i=1}^{|\mathbf{X}_v|} H(y_i, p_i) \quad (11)$$

where $H(\cdot)$ denotes the cross-entropy loss, and y_i, p_i is the ground truth and predicted label of $\hat{x}_i$ respectively.

$\mathcal{L}_L$ is derivable with respect to the parameters θ_G of the generator G. According to the chain rule, we have

$$\frac{\partial \mathcal{L}_L}{\partial \theta_G} = \frac{\partial \mathcal{L}_L}{\partial \theta_L} \cdot \frac{\partial \theta_L}{\partial \mathbf{X}_\eta} \cdot \frac{\partial \mathbf{X}_\eta}{\partial \theta_G}. \quad (12)$$

Here, θ_L denotes the parameter of L. That is, we can teach G to generate training data (i.e., to impute missing values in training data) that enables the learner to perform well in the validation set (observed data). If the distribution of $\mathbf{X}$ and test data in the target task is similar (an assumption often made when applying machine learning models), we can guarantee the utility of our imputed data for training the downstream model. Meanwhile, the learner can contribute to keeping the same data distribution between the imputed and observed data.

Several considerations arise in the design of this plug-in. First, it will be costly to continuously train a new classifier for each generated data instance. As a result, we incorporate $\mathcal{L}_L$ only after the diffusion process is completely finished. Second, in the case of a high rate of missing data, it becomes difficult to find tuples that do not contain missing values for the validation set. In this situation, we utilize the discriminator D to construct the most appropriate validation set. Concretely, we assign a confidence score to each tuple in $\hat{\mathbf{X}}$: for each observed attribute value, it has a confidence score of 1, whereas each missing attribute value receives a score derived from D, i.e., the probability that this value is enough to pass off as real; the confidence score for a tuple is the sum of scores across all its attributes. We then select some top of the tuples after sorting in descending order of their confidence score to compose the validation set.

V. DATA DEPENDENCY PLUG-IN

A. Discovery of Functional Dependencies from Missing Data

Berti-Équille et al. [3] have summarized previous work for discovering FDs from data with missing values and mentioned their respective drawbacks. For example, using NULL semantics or directly skipping all tuples with missing values will make many correct FDs undiscoverable. To avoid this, given an FD $\varphi : Z \rightarrow A$, we expect to verify the soundness of φ using all tuples that do not contain missing values on $Z \cup \{A\}$, rather than skipping all tuples with missing values on any of the attributes U .

We adapt TANE [15] to achieve this goal. TANE introduces the notation of a *partition*, denoted by π . For a set of

attributes Z , π_Z is a collection of *equivalence classes* of tuples, such that each class has a unique value for the attribute set Z . We redefine the partition. Specifically, given a set of attributes Z , we omit the missing data to calculate π_Z . Thus, the union of these equivalence classes in π_Z equals a set of tuples with no missing value in Z . For example, $\pi_{\text{AreaCode}} = \{\{t_1, t_2, t_3, t_4\}, \{t_6, t_7\}, \{t_8\}\}$. Then, we redefine the formulation for deriving $\pi_{Z \cup \{A\}}$ from π_Z and $\pi_{\{A\}}$ under the new definition of partition:

$$\pi_{Z \cup \{A\}} = (\cup_{z_i \in \pi_Z} (z_i \cap (\cup_{a_j \in \pi_{\{A\}}} a_j))) \setminus \{\emptyset\} \quad (13)$$

This remove tuples from $\pi_{Z \cup \{A\}}$ that contain missing values on attribute Z or A . Under the above setting, $Z \rightarrow A$ holds if and only if for each $z_i \in \pi_Z$, there exists at most one subset of z_i in $\pi_{Z \cup \{A\}}$.

We set a threshold τ to limit the maximum number of attributes probed on the left-hand side of an FD. The reason is that when considering an excessive number of attribute combinations, the FD holds mostly because there are no two tuples with the same value in attributes on the left-hand side, which does not work for reasoning about missing values. Moreover, we will prune some of the candidate FDs during the mining process. For example, if $Z \rightarrow A$ holds, we will not consider $Z \cup Z' \rightarrow A$ unless $Z \rightarrow A$ is found to be invalid in the following phase of refinement. We also filter out FDs that contain the primary key on the left-hand side, which does not contribute to missing value imputation. Thus, the number of potential FDs mined from the incomplete tabular data is not particularly large. This approach might inadvertently lead to the discovery of some erroneous FDs. The set F of potential FDs mined from X may differ from those $F^\vee$ held in $X^\vee$. Thus, they will be refined based on the imputed data in the subsequent process, illustrated in Section V-C, where most of the erroneous FDs can be rectified in a few iterations.

B. Modeling of Functional Dependencies

Given a set F of potential FDs mined from X , we build a dependency model Φ_φ for each FD $\varphi \in F : Z \rightarrow A$, which takes the attribute values of Z as input and predicts the attribute value of A . Thus, we utilize all tuples with no missing value in $Z \cup \{A\}$ as training data for Φ_φ , and for each tuple, their attribute values of Z and A are taken as features and the label, respectively. Actually, these dependency models could also be pre-trained using the master data or knowledge bases after alignment. This allows more external information to guide the imputation of missing values.

It is worth noting that the dependency model Φ_φ actually accepts values in all attributes $U \setminus \{A\}$ as input, not just in Z . But all attribute values except Z are set to zero for input to the model. That means we manually perform a “dropout” operation that ignores the inputs in the other attributes $U \setminus (Z \cup \{A\})$, but still retains the nodes on the input layer of Φ_φ . The reason behind this is that as the imputation becomes more accurate, φ might be proven to be incorrect. Thus we need to add more attributes to the left-hand side of φ to make it valid (more details can be found in Section V-C). In such a case, we

need to “activate” the corresponding input nodes and fine-tune the model instead of training a new dependency model, which will save a lot of training costs without sacrificing the model’s performance.

Then, given the imputed table $\bar{X}$, we expect it to satisfy all the FDs in F . That is, the values imputed by the generator G are desired to be consistent with the predictions of the dependency models. Let $\mathbf{R}_F = \{A_{\varphi_1}, \dots, A_{\varphi_{|F|}}\}$ where A_{φ_i} be the attribute in the right-hand side of the i th FD in F . Then, we define the following loss for the generator:

$$\mathcal{L}_F = \sum_{i=1}^n \sum_{j=1}^{|F|} \|\bar{\mathbf{v}}_{ij} - \mathbf{v}_{ij}\|_1 \quad (14)$$

where $\|\cdot\|_1$ denotes the L1-norm as the distance measure between two vectors; $\bar{\mathbf{v}}_{ij}$ and $\mathbf{v}_{ij}$ are representations of the attribute value of the i th tuple in attribute A_{φ_j} given by the generator G and corresponding dependency model Φ_{φ_j} . In fact, since we take one-hot encoding for the categorical attributes in the data imputation module, $\bar{\mathbf{v}}_{ij}$ and $\mathbf{v}_{ij}$ are predicted probability vectors given by the model G and Φ_{φ_j} respectively.

C. Inaccurate FDs Refinement

When mining the potential FDs, we ignore the missing data, assuming that tuples containing missing values always satisfy the mined FDs, which is clearly flawed. As the imputation becomes more accurate, we must go back to determine whether these FDs are valid. In this section, we introduce a framework to refine inaccurate FDs based on the confidence of data and constraints.

Confidence of Tuple. The confidence of a tuple is determined by the confidence of its attribute values. Given an FD $\varphi : Z \rightarrow A$, the confidence of a tuple $\bar{x}_i$ under φ is the minimum value of the confidence of attribute values in $Z \cup \{A\}$:

$$c_\varphi(\bar{x}_i) = \min(c(\bar{x}_{ij})), A_j \in Z \cup \{A\} \quad (15)$$

Confidence of attribute value. It goes without saying that the confidence of observed data should be 1. As for the missing data, we derive the confidence of imputed value $\bar{x}_{ij}$ based on the predictions of generator G and discriminator D . As stated earlier, we adopt one-hot encoding for the categorical attribute. Therefore, for the j th attribute of the i th tuple, the output of G is a predicted probability vector, and $\bar{x}_{ij}$ is the value with the highest probability, which is denoted by $p_G(x_{ij})$ and can also be taken as the confidence of imputed value $\bar{x}_{ij}$ given by G . D gives the probability $p_D(x_{ij})$ that $\bar{x}_{ij}$ turns out to be observed. The larger $p_D(x_{ij})$, the more $\bar{x}_{ij}$ looks like being real. We have also found from the experiment that $p_D(x_{ij})$ being larger means x_{ij} is imputed correctly. Thus, we take it as the confidence of imputed value $\bar{x}_{ij}$ provided by D . In brief, the confidence of $\bar{x}_{ij}$ is the weighted sum of $p_G(x_{ij})$ and $p_D(x_{ij})$:

$$c(\bar{x}_{ij}) = \begin{cases} 1, & m_{ij} = 1 \\ \frac{w_G \times p_G(x_{ij}) + w_D \times p_D(x_{ij})}{w_G + w_D}, & m_{ij} = 0 \end{cases} \quad (16)$$

Here w_G and w_D are the accuracy of G and D measured from the observed data. Concretely, w_G is the proportion of observed values G correctly imputed among all observed values. The more values G can correctly impute, the more reliable the probability $p_G(x_{ij})$ it gives. w_D is the sum of the predicted probabilities over the observed values divided by the total number of the observed values. The larger w_D is, the better D is at recognizing observed values, and the more reliable the probability $p_D(x_{ij})$ it gives. **Note that we can choose alternative encoding methods for categorical attributes, such as learned embeddings. In this case, the confidence of $\bar{x}_{ij}$ can be measured solely by $p_D(x_{ij})$.**

Example 1: Given the FD $\varphi_3 : \text{AreaCode} \rightarrow \text{SingleExemp}$, let us compute the confidence of t_5 under φ_3 . Suppose the predicted probability vector given by G for $t_5[\text{AreaCode}]$ is (0.05, 0.8, 0.15), and the output from D is 0.9. There are totally 42 observed data in all categorical attributes. Let us say G correctly predicts 32 of them, and the sum of predicted probabilities by D is 38.64. We have $w_G = 32/42 \approx 0.76$ and $w_D = 38.64/42 \approx 0.92$. The confidence of $t_5[\text{AreaCode}]$ is $(0.76 \times 0.8 + 0.92 \times 0.9)/(0.76 + 0.92) \approx 0.85$. Subsequently, we can determine the confidence of t_5 under φ_3 as $\min(0.85, 1) = 0.85$.

Confidence of FD. Following the previous work [3], we construct a subset of tuples $\bar{X}_\varphi$ from $\bar{X}$ that satisfy φ to measure the confidence $c(\varphi)$ of an FD $\varphi : Z \rightarrow A$. To avoid using erroneous tuples to measure the confidence of an FD, we choose tuples with higher confidence, which is consistent with the intuition that the higher the confidence of tuples that satisfy a constraint, the more likely the constraint is correct.

$\bar{X}_\varphi$ should initially contain all tuples with no missing value in attributes $Z \cup \{A\}$ (The FD φ is extracted from these tuples, so they are naturally supposed to serve as evidence that φ is valid). For the other tuples, we sort them in descending order of their confidence and add them in turn to $\bar{X}_\varphi$. Note that if a tuple has already been in $\bar{X}_\varphi$, tuples that have violation with it *w.r.t.* φ should be excluded. Finally, we have

$$c(\varphi) = \frac{\sum_{\bar{x}_i \in \bar{X}_\varphi} c_\varphi(\bar{x}_i)}{\sum_{\bar{x}_j \in \bar{X}} c_\varphi(\bar{x}_j)}. \quad (17)$$

It is worth noting that tuples with unique values in attributes Z are not used to derive the confidence of φ since they do not play any role in determining whether the FD holds. Thus, we remove them from $\bar{X}_\varphi$ and $\bar{X}$ before calculating the confidence of rule φ .

Example 2: Continue to consider φ_3 in Example 1. Suppose $t_5[\text{Areacode}]$ and $t_3[\text{SingleExemp}]$ are correctly imputed with “501” and “1000” respectively, and the confidence of t_3 is 0.67. We can find from the imputed table that t_3 have the same value with $\{t_1, t_2, t_4\}$ in AreaCode but different in SingleExemp. Given that $\{t_1, t_2, t_4\}$ under φ_3 are all observed, we put $\{t_1, t_2, t_4\}$ in $\bar{X}_{\varphi_3}$. Similarly, $\{t_6, t_7\}$ are also added to $\bar{X}_{\varphi_3}$. We exclude t_8 since it has a distinct value of AreaCode. Then, the confidence of φ_3 is calculated as $c(\varphi_3) = (1 + 1 + 1 + 1 + 1)/(1 + 1 + 0.67 + 1 + 0.85 + 1 + 1) \approx 0.77$.

FD Refinement. We compare the confidence of a constraint and tuples violating it to determine whether an FD φ is valid, *i.e.*, $\varphi \in F^\vee$. Concretely, the confidence of φ can be calculated by Equation 17. In the meantime, we choose the tuple $\bar{x}_\varphi^*$ from $\bar{X} \setminus \bar{X}_\varphi$ with the highest confidence as an indicator that φ may be invalid. It stems from the notion that higher confidence indicates the tuple $\bar{x}_\varphi^*$ is closer to its true value, and φ obviously cannot hold on $\bar{X}_\varphi \cup \{\bar{x}_\varphi^*\}$. When the FD φ has higher confidence than $\bar{x}_\varphi^*$, we continue to employ it to guide the imputation of missing values. Otherwise, φ is considered inaccurate and necessitates a refinement. Here, we can find the calculation of FD confidence in Equation 17 indirectly improves the confidence of the violated tuples required for the refutation of FD, thus ensuring that only a correct pair of tuples conflicting on the FD can give the conclusion that it is inaccurate.

Before discussing specific strategies of FD refinement, we first analyze what changes F has to make to become $F^\vee$. There are four optional steps: (1) add a new FD φ' that $\varphi' \in F^\vee$; (2) remove an FD φ from F that $\varphi \notin F^\vee$; (3) add some new attributes to the left-hand side of $\varphi \in F$; (4) remove some attributes from the left-hand side of $\varphi \in F$. Choice 1 is only required if the missing rate is very high (experimentally found to be $>85\%$). This is uncommon and what we can do is re-mining the FDs from $\bar{X}$. Thus, we will not discuss this scenario in detail. Choice 4 will never be adopted, and that is because if φ does not hold, then it cannot be valid after removing an attribute from the left-hand side of φ . Hence, we only consider choice 2 and 3, *i.e.*, when the confidence of an FD is less than the violated tuple, we add some attributes to its left-hand side. If the number of attributes in the left-hand side after appending exceeds the threshold τ , we discard this FD when it still does not hold.

When appending attributes to $\varphi : Z \rightarrow A$, we need the power set of $U \setminus (Z \cup \{A\})$ (excluding $\emptyset$ and sets larger than $\tau - |Z|$). Each item in the power set is taken in ascending order of the size into consideration whether it can be appended to the left-hand side of φ . Here, if appending an attribute set Z' yields an FD $\varphi' : Z \cup Z' \rightarrow A$, and $c(\varphi') \geq c(\bar{x}_\varphi^*)$, then no more super sets of Z' are considered. We discard φ if it still cannot hold after trying all the possible appends.

Retraining of Dependency Model. If more attributes are added to the left-hand side of FD φ (φ is refined to $\varphi' : Z \cup Z' \rightarrow A$), we need to retrain the dependency model Φ_φ after enabling the input nodes of attributes Z' to obtain a new model $\Phi_{\varphi'}$. That is, we utilize all tuples in $\bar{X}_{\varphi'}$ to retrain Φ_φ : for each tuple, their attribute values of $Z \cup Z'$ and A are taken as features and the label, respectively, and all attribute values except $Z \cup Z'$ still need to be set to zero for input to the model. Since Φ_φ has already captured some inner connection between Z and A , the retraining mainly goes to determine the effect of the newly added attributes on A , which will not take long. If the FD φ is entirely discarded, the corresponding dependency model Φ_φ is also eliminated with it.

Algorithm 1: Training Process of DDGANI

Input: relational table X with missing values, mask M , the number of diffusion steps T , input noise z , hyper-parameters such as loss weights α, β

Output: imputed relational table $\bar{X}$

- 1 $X' \leftarrow$ pre-processing X with the self-attention mechanism;
- 2 initialize the parameters of generator G and discriminator D ;
- 3 **extract potential FDs F from X and build their corresponding dependency models;**
- 4 **repeat**
- 5 $\{X'_1, \dots, X'_T\} \leftarrow$ add noise to X' over T steps;
- 6 **for** step $t \in [T, \dots, 1]$ **do**
- 7 $\hat{X}_0 \leftarrow G(X'_t, z, t)$;
- 8 compute the reconstruction loss $\mathcal{L}_R$ with $\hat{X}_0$ and X' ;
- 9 $\bar{X}_0 \leftarrow (1 - M) \odot \hat{X}_0 + M \odot X'$;
- 10 **compute the data dependency loss $\mathcal{L}_F$ with $\bar{X}_0$;**
- 11 $\hat{X}_{t-1} \leftarrow$ posterior sampling with $\hat{X}_0$ and X'_t ;
- 12 $\bar{X}_{t-1} \leftarrow (1 - M) \odot \hat{X}_{t-1} + M \odot X'_{t-1}$;
- 13 $\bar{M} \leftarrow D(X'_t, \bar{X}_{t-1}, t)$;
- 14 compute the discrimination loss $\mathcal{L}_D$ and $\mathcal{L}_M$;
- 15 $\mathcal{L}_G \leftarrow \mathcal{L}_R + \alpha\mathcal{L}_M + \beta\mathcal{L}_F$;
- 16 update the parameters of G , D by $\mathcal{L}_G, \mathcal{L}_D$ respectively;
- 17 **refine F based on $\bar{X}_0$ and retrain the dependency models;**
- 18 $\bar{X}_v \leftarrow$ randomly pick some tuples from $\bar{X}_0$ whose attribute values are all observable;
- 19 train the learner L with the rest tuples $\bar{X}_0 \setminus \bar{X}_v$;
- 20 evaluate L on $\bar{X}_v$ and compute $\mathcal{L}_L$;
- 21 update the parameters of G by $\mathcal{L}_L$;
- 22 **until** convergence of G and D ;
- 23 **return** $\bar{X}_0$

VI. ENTIRE TRAINING PROCESS OF DDGANI

The overall workflow of DDGANI is presented in Algorithm 1. Let us first focus on the data imputation module, which is in black. The missing data is initially pre-processed with the attention mechanism to obtain X' (line 1). We then initialize the parameters of generator G and discriminator D (line 2) and train the models (lines 4-22). A forward diffusion process is applied on X' in the beginning of each epoch to produce $\{X'_1, \dots, X'_T\}$ (line 5). Then in each step t of denoising, G takes X'_t , t and a random Gaussian noise z as input and outputs $\hat{X}_0$ (line 7). The loss $\mathcal{L}_R$ is computed based on the differences between $\hat{X}_0$ and X' (line 8). Then, a posterior sampling process derives $\hat{X}_{t-1}$ from $\hat{X}_0$ and X'_t (line 11). We integrate $\hat{X}_{t-1}$ and X'_{t-1} to obtain the final imputed table $\bar{X}_{t-1}$ at step $t-1$, which is fed into D with X'_t and $t-1$ to predict the mask M (lines 12-13). The differences between the predicted result and M will be used to compute the loss

TABLE II: Dataset Description

Dataset	#Instance	#Numerical	#Categorical	#FDs
Wine	178	13	0	0
Spam	4,601	57	0	0
Tic-Tac-Toe	958	0	9	0
Flare	1,066	0	12	0
Adult	30,148	5	9	2
Food	50,000	2	7	5
Hospital	1,000	2	7	8
Tax	100,000	2	7	9
Contraceptive	1,473	2	7	0
Australian	690	6	8	0

$\mathcal{L}_D$ and $\mathcal{L}_M$, which are used to update the parameters of D and G along with $\mathcal{L}_R$, respectively (lines 14-16).

Then, let us look at the data utility plug-in, which is in gray. After the data imputation module has finished its work, we randomly draw some tuples without missing values from $\bar{X}_0$ as the validation set $\bar{X}_v$, and a learner L is trained using the remaining data (lines 18-19). $\mathcal{L}_L$ is the validation loss of L , which will be used to update the parameters of G via meta-learning paradigm (lines 20-21).

The data dependency plugin is in brown. We will first mine potential FDs at the beginning and build their corresponding dependency models (line 3). After obtaining $\bar{X}_0$ at each step t , we combine $\bar{X}_0$ and X' , yielding the imputed data $\bar{X}_0$ and calculating the loss $\mathcal{L}_F$ (lines 9-10). This loss will be added into $\mathcal{L}_G$ to update the parameters of G (line 15). These FDs may be refined according to the confidence of tuples and constraints after the missing value imputation (line 17).

VII. EXPERIMENTS

A. Experimental Settings

Dataset. We evaluate our approach on **ten** publicly available datasets, including two datasets with all numerical attributes *Wine*, *Spam*, two datasets with all categorical attributes *Tic-Tac-Toe*, *Flare*, and **six** datasets with both numerical and categorical attributes *Adult*, **Food** [30], *Hospital* [30], *Tax* [5], *Contraceptive*, *Australian*. Datasets not specifically attributed are from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/>). Table II presents an overview of these datasets, including the number of tuples, numerical attributes, categorical attributes, and functional dependencies. Here, the FDs are mined by TANE [15] from the complete dataset and treated as the true FDs in Section VII-G.

Each dataset is randomly divided into three parts: 70% of the data with missing values injected to evaluate the accuracy of the imputation methods; 20% of the data, which remains complete and is used to report the accuracy of the downstream classifier; 10% of the data for the hyperparameter search.

Missing Value Injection. We remove some attribute values from the datasets completely at random with probability p (*Missing Completely at Random, MCAR*). The missing rate p is set to 20% by default and is changed from 10% to 50% in Section VII-C to show the effectiveness of our proposed method with different missing rates. Section VII-D also shows the evaluation of our method under *MAR* (*Missing at Random*)

TABLE III: Overall Evaluation of the Model (Average $\pm$ Std)

Dataset	Metrics	MissFI	AimNet	TabImpute	PMAE	UnIMP	DiffPuter	TabDiff	DDGANI
Wine	ARMSE	0.151 $\pm$ 0.003	0.241 $\pm$ 0.014	0.153 $\pm$ 0.001	0.157 $\pm$ 0.004	0.132$\pm$0.003	0.179 $\pm$ 0.012	0.154 $\pm$ 0.006	0.145 $\pm$ 0.001
	AMAE	0.113 $\pm$ 0.002	0.185 $\pm$ 0.010	0.117 $\pm$ 0.001	0.121 $\pm$ 0.004	<u>0.110$\pm$0.003</u>	0.126 $\pm$ 0.013	0.116 $\pm$ 0.009	0.106$\pm$0.001
Spam	ARMSE	<u>0.048$\pm$0.000</u>	0.051 $\pm$ 0.007	0.054 $\pm$ 0.004	0.050 $\pm$ 0.000	<u>0.048$\pm$0.002</u>	0.050 $\pm$ 0.005	0.049 $\pm$ 0.002	0.047$\pm$0.000
	AMAE	0.023 $\pm$ 0.000	0.022 $\pm$ 0.001	0.021 $\pm$ 0.003	0.021$\pm$0.000	0.022 $\pm$ 0.002	0.021 $\pm$ 0.002	0.021 $\pm$ 0.001	0.021$\pm$0.000
Tic-Tac-Toe	ARMSE	0.529 $\pm$ 0.008	0.525 $\pm$ 0.007	0.533 $\pm$ 0.004	0.521 $\pm$ 0.004	0.518 $\pm$ 0.005	0.526 $\pm$ 0.002	<u>0.516$\pm$0.012</u>	0.512$\pm$0.002
	AMAE	0.529 $\pm$ 0.008	0.525 $\pm$ 0.007	0.533 $\pm$ 0.004	0.521 $\pm$ 0.004	0.518 $\pm$ 0.005	0.526 $\pm$ 0.002	<u>0.516$\pm$0.012</u>	0.512$\pm$0.002
Flare	ARMSE	0.240 $\pm$ 0.000	0.241 $\pm$ 0.002	0.257 $\pm$ 0.003	0.206 $\pm$ 0.005	<u>0.186$\pm$0.004</u>	0.215 $\pm$ 0.007	0.214 $\pm$ 0.003	0.173$\pm$0.003
	AMAE	0.240 $\pm$ 0.000	0.241 $\pm$ 0.002	0.257 $\pm$ 0.003	0.206 $\pm$ 0.005	<u>0.186$\pm$0.004</u>	0.215 $\pm$ 0.007	0.214 $\pm$ 0.003	0.173$\pm$0.003
Adult	ARMSE	0.258 $\pm$ 0.000	0.293 $\pm$ 0.001	0.236 $\pm$ 0.006	0.239 $\pm$ 0.014	<u>0.213$\pm$0.013</u>	0.261 $\pm$ 0.013	0.218 $\pm$ 0.021	0.195$\pm$0.000
	AMAE	0.244 $\pm$ 0.000	0.272 $\pm$ 0.001	0.216 $\pm$ 0.006	0.228 $\pm$ 0.011	<u>0.196$\pm$0.008</u>	0.205 $\pm$ 0.011	0.198 $\pm$ 0.024	0.180$\pm$0.000
Food	ARMSE	<u>0.689$\pm$0.013</u>	0.632 $\pm$ 0.013	0.626 $\pm$ 0.005	0.617 $\pm$ 0.008	<u>0.596$\pm$0.011</u>	0.608 $\pm$ 0.008	0.599 $\pm$ 0.013	0.589$\pm$0.004
	AMAE	<u>0.683$\pm$0.010</u>	0.625 $\pm$ 0.013	0.623 $\pm$ 0.005	0.613 $\pm$ 0.006	<u>0.590$\pm$0.011</u>	0.604 $\pm$ 0.008	0.592 $\pm$ 0.013	0.584$\pm$0.004
Hospital	ARMSE	0.254 $\pm$ 0.003	0.169 $\pm$ 0.003	0.294 $\pm$ 0.001	0.246 $\pm$ 0.004	<u>0.142$\pm$0.005</u>	0.233 $\pm$ 0.003	0.205 $\pm$ 0.020	0.133$\pm$0.002
	AMAE	0.239 $\pm$ 0.003	0.139 $\pm$ 0.001	0.281 $\pm$ 0.001	0.231 $\pm$ 0.004	<u>0.125$\pm$0.006</u>	0.224 $\pm$ 0.003	0.196 $\pm$ 0.021	0.115$\pm$0.002
Tax	ARMSE	0.380 $\pm$ 0.011	0.422 $\pm$ 0.026	0.283 $\pm$ 0.040	0.299 $\pm$ 0.030	<u>0.266$\pm$0.002</u>	0.303 $\pm$ 0.002	0.278 $\pm$ 0.009	0.234$\pm$0.000
	AMAE	0.360 $\pm$ 0.009	0.391 $\pm$ 0.026	0.262 $\pm$ 0.037	0.279 $\pm$ 0.030	<u>0.234$\pm$0.003</u>	0.286 $\pm$ 0.002	0.244 $\pm$ 0.010	0.212$\pm$0.000
Contraceptive	ARMSE	0.328 $\pm$ 0.003	0.337 $\pm$ 0.003	<u>0.305$\pm$0.009</u>	0.324 $\pm$ 0.005	0.314 $\pm$ 0.015	0.308 $\pm$ 0.007	0.308 $\pm$ 0.014	0.290$\pm$0.001
	AMAE	0.318 $\pm$ 0.002	0.328 $\pm$ 0.003	0.296 $\pm$ 0.008	0.315 $\pm$ 0.005	0.302 $\pm$ 0.011	0.293 $\pm$ 0.008	0.295 $\pm$ 0.014	0.281$\pm$0.001
Australian	ARMSE	0.269 $\pm$ 0.013	0.255 $\pm$ 0.006	0.260 $\pm$ 0.005	0.266 $\pm$ 0.003	0.257 $\pm$ 0.006	<u>0.245$\pm$0.006</u>	0.267 $\pm$ 0.008	0.237$\pm$0.003
	AMAE	0.254 $\pm$ 0.014	<u>0.232$\pm$0.004</u>	0.238 $\pm$ 0.005	0.248 $\pm$ 0.003	0.239 $\pm$ 0.006	0.232 $\pm$ 0.007	0.244 $\pm$ 0.007	0.226$\pm$0.002

and MNAR (Missing Not at Random). More details can be found in Section VII-D.

Evaluation Metrics. We employ two commonly used metrics to measure the accuracy of imputation methods: *average root mean square error* (ARMSE) [35] and *average mean absolute error* (AMAE) [28]. ARMSE calculates the average of the errors for each attribute. Specifically, it utilizes the *root mean square error* (RMSE) [18] for numerical attributes and the *accuracy error* (AR) [28] for categorical ones. The smaller ARMSE corresponds to the better imputation result. Similarly to ARMSE, AMAE employs *mean absolute error* (MAE) [18] for numerical attributes and AR for categorical ones. The smaller AMAE corresponds to the better imputation result. We also measure the *accuracy* of the downstream classifier in some experiments to verify the utility of the imputed dataset.

Baseline. We compare DDGANI with the following data imputation methods. (1) *MissFI* [35], which utilizes the missing forest to impute the missing data; (2) *AimNet* [41], which utilizes a transformer to reconstruct missing data and is also based on FDs and the attention scores between attributes; (3) *TabImpute* [12], which utilizes a pre-trained neural network that exploits learned representations to predict missing values from correlated attributes; (4) *PMAE* [20], which extends the masked autoencoding framework to impute missing values of tabular data; (5) *UnIMP* [39], which is a unified framework that leverages LLM and high-order message passing to enhance the imputation of mixed-type data; (6) *DiffPuter* [44], which is based on denoising diffusion probabilistic model that imputes missing values by iterative denoising conditioned on observed values; (7) *TabDiff* [33], which develops a principled diffusion model for generating mixed-type tabular data, also imputing the missing values.

Hyperparameter Settings. We set the diffusion timestep T at 6, the threshold κ for the attention mechanism to 0.05,

and the limit τ for the number of attributes on the left-hand side of FDs to 3. The diffusion rate β_t at step t is defined to be linearly increasing constants between $\beta_1 = 10^{-4}$ and $\beta_T = 2 \times 10^{-2}$. Other hyperparameters in DDGANI and each competing method are decided via cross-validation.

Experimental Environment. All experiments were conducted on an Intel(R) Xeon(R) Silver 4210R 2.40GHz server with NVIDIA GeForce RTX 4090 (GPU).

B. Overall Evaluation of the Model

We first evaluate the performance of DDGANI in **ten** datasets with a missing rate of 20% in the MCAR model. The results are presented in Table III. We conduct five runs for each method and demonstrate the average and standard deviation of the results. The best scores are in bold, and the second-best scores are underlined. The results of our method are on a gray background.

Table III shows that DDGANI outperforms other methods **in most cases**, which can be attributed to the way DDGANI approaches the target distribution through a series of incremental approximations. This iterative approach focuses on refining the current distribution rather than learning a distribution from scratch, allowing the model to better capture subtle nuances and intricate dependencies within the data. Additionally, the utilization of the attention mechanism and two plug-ins also helps the model better learn the data distribution and the data dependencies. Note that DDGANI could be expanded to support CFDs. We find that employing CFDs can further enhance the quality of imputation. For example, the replacement of FDs with CFDs reduces the ARMSE of DDGANI from 0.589 $\pm$ 0.004 to 0.554 $\pm$ 0.012 and MAE from 0.584 $\pm$ 0.004 to 0.551 $\pm$ 0.012 in the Food dataset.

AimNet is also based on the attention mechanism for missing value imputation. However, it only employs a schema-level

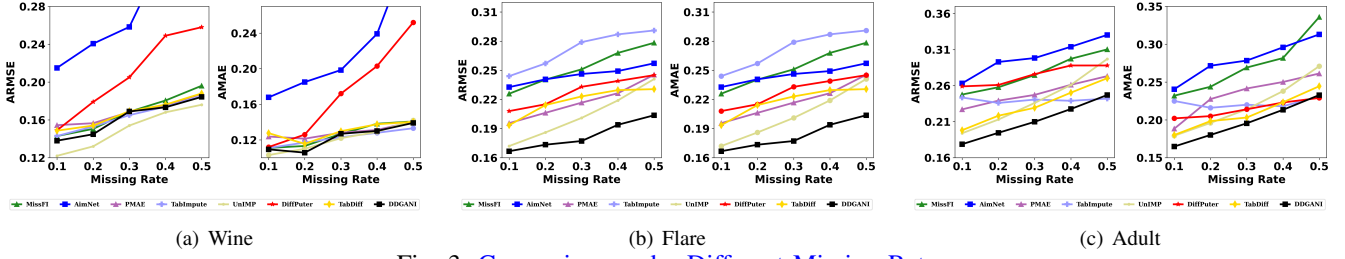


Fig. 3: Comparison under Different Missing Rates

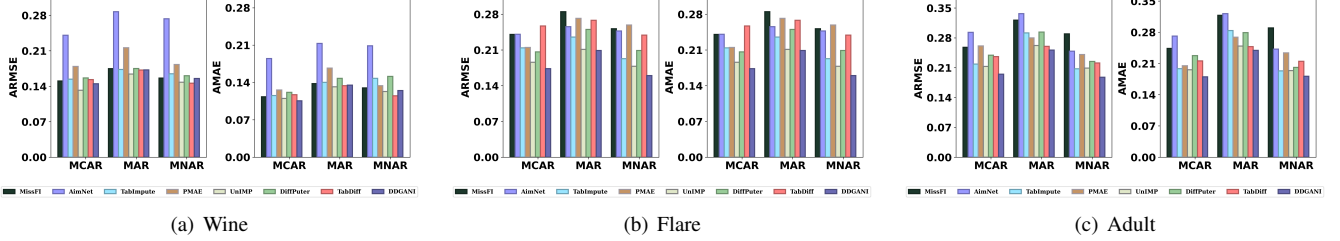


Fig. 4: Comparison under Different Missing Types

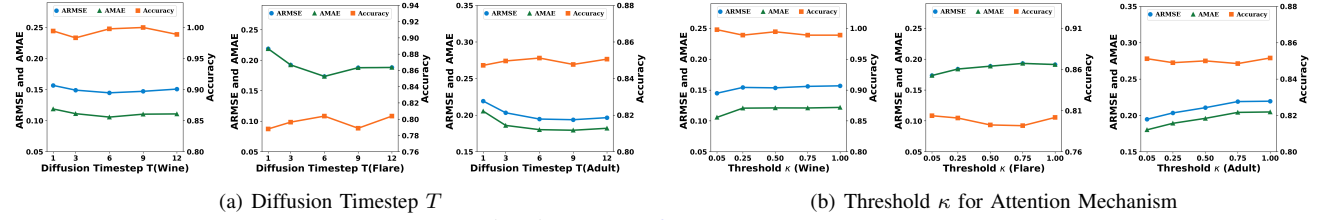


Fig. 5: Impact of Hyperparameters

attention mechanism, which tightly embeds into its prediction network, while DDGANI adopts a model-agnostic tuple-level attention mechanism, combining attribute-level correlations, thus fully utilizing the information provided from similar tuples to impute the missing values.

TabImpute is trained on synthetic data generated from a linear factor model, which implicitly assumes a low-rank structure with continuous latent variables. This assumption limits TabImpute’s performance on datasets dominated by categorical attributes or featuring discrete structures induced by data constraints. UnIMP captures high-order dependencies through message passing, which enables strong robustness and consistently competitive performance under diverse missing patterns. The utilization of LLM grants UnIMP a clear advantage when natural language semantics can be effectively exploited. However, since the model is optimized for conditional prediction, complex high-order dependencies can only be captured implicitly. DDGANI explicitly models inter-column dependencies and optimizes the joint data distribution, which leads to better performance on datasets with complex distributions.

PMAE performs one-shot reconstruction of missing values in a single forward pass, which is hard to model uncertainty and refine predictions. DiffPater integrates the diffusion model into an EM framework. While theoretically appealing, this design can amplify early imputation errors (e.g., mean biases), especially under high missing ratios. Furthermore, although DiffPater ensures consistency with observed data through conditional score matching, it also lacks an explicit mechanism to evaluate the realism of imputed tuples as a whole. In contrast,

our DDGANI introduces a discriminator that evaluates the realism of imputed tuples holistically. The attention module, combined with the data dependency plugin, enhances the modeling of complex long-range feature interactions, thereby generating more globally coherent imputation results. The inferior performance of TabDiff similarly reveals this.

C. Comparison under Different Missing Rates

To better showcase the effectiveness of DDGANI, we vary the missing rate from 10% to 50% to test the performance of each method. In this set of experiments, we still choose Wine, Flare, and Adult as the representatives of the datasets. The results are given in Figure 3.

Obviously, as the missing rate increases, the performance of each imputation method declines. This is because the reduction in observed data makes it difficult to estimate the missing values, especially when the data near the missing values is gradually unavailable. Nevertheless, DDGANI consistently performs better than other baselines. The reason behind this is that DDGANI employs the diffusion model, which is more effective at predicting missing values based on context through several conditional denoising diffusion steps. Meanwhile, the usage of self-attention mechanisms and data dependency plugin allows the model to better capture the correlation between missing values and the information of observed data.

D. Comparison under Different Missing Types

In this set of experiments, we evaluate the impact of various missing mechanisms on the performance of imputation methods. According to a recent study [16], we inject different types

TABLE IV: Accuracy of Downstream Classifier

Dataset	MissFI	AimNet	TabImpute	PMAE	UnIMP	DiffPuter	TabDiff	DDGANI	Removal	GroundTruth
Wine	99.44±1.11	93.52±1.31	99.44±0.01	99.54±1.11	99.76±0.02	96.67±0.09	98.45±1.24	99.78±0.12	88.89±0.00	100.00±0.00
Spam	93.25±0.00	92.67±2.36	94.08±0.03	94.02±0.05	94.26±0.00	94.21±0.22	93.90±0.03	94.81±0.03	-	95.55±0.00
Tic-Tac-Toe	83.44±0.88	83.68±0.49	83.65±0.00	87.63±0.88	88.31±0.15	87.60±0.04	88.36±0.72	88.72±1.30	77.92±0.00	98.85±0.00
Flare	80.06±0.79	77.57±1.01	79.81±0.01	79.57±0.80	80.30±1.25	79.50±0.05	79.41±0.62	80.37±1.10	78.60±0.00	81.79±0.00
Adult	85.00±0.05	82.62±0.43	83.35±0.02	84.96±0.08	84.98±0.04	84.85±0.04	85.04±0.03	85.12±0.09	81.43±0.00	85.34±0.00
Food	76.00±0.20	76.86±0.03	75.50±0.08	77.25±0.09	77.60±0.03	78.00±0.13	78.00±0.05	78.98±0.11	76.10±0.00	82.02±0.00
Hospital	96.50±0.00	96.83±0.24	96.50±0.00	96.90±0.00	96.81±0.14	96.60±0.02	96.80±0.00	97.00±0.00	95.00±0.00	97.00±0.00
Tax	87.04±0.82	84.91±0.37	86.87±0.07	87.65±0.45	87.69±0.46	87.84±0.45	87.32±0.36	88.73±0.66	86.29±0.00	89.41±0.00
Contraceptive	52.09±0.32	49.27±1.76	52.68±0.00	54.64±0.45	55.02±0.22	53.10±0.04	54.46±0.32	55.59±0.36	49.56±0.00	55.98±0.00
Australian	86.71±0.34	87.20±0.49	87.83±0.00	87.83±0.12	87.98±0.15	85.54±0.05	88.01±0.22	88.13±0.29	82.90±0.00	89.86±0.00

TABLE V: An Ablation Study

Dataset	Metrics	DDGANI	w/o $\mathcal{L}_L$	w/o $\mathcal{L}_F$	w/o attention
Wine	ARMSE	0.145±0.001	0.157±0.002	-	0.164±0.012
	AMAE	0.106±0.001	0.121±0.002	-	0.123±0.009
	Accuracy	99.78±0.12	98.33±0.22	-	99.21±0.02
Flare	ARMSE	0.173±0.003	0.194±0.002	-	0.232±0.002
	AMAE	0.173±0.003	0.194±0.002	-	0.232±0.002
	Accuracy	80.37±1.10	78.88±1.16	-	77.94±0.01
Adult	ARMSE	0.195±0.000	0.206±0.000	0.201±0.002	0.248±0.009
	AMAE	0.180±0.000	0.192±0.000	0.190±0.001	0.236±0.009
	Accuracy	85.12±0.09	84.58±0.09	84.96±0.13	82.10±0.01
Tax	ARMSE	0.349±0.004	0.360±0.006	0.366±0.004	0.375±0.024
	AMAE	0.325±0.003	0.340±0.004	0.345±0.001	0.352±0.024
	Accuracy	88.73±0.66	86.70±0.30	87.62±0.62	66.87±0.06

of missing values. 1) MCAR: removing some data from the dataset completely at random; 2) MAR: randomly selecting an attribute to sort the tuples and then selecting some consecutive tuples to inject missing values into their other attributes; 3) MNAR: randomly removing two types of attribute values; one is the data in a numerical attribute that is less than the median, and the other is from a categorical attribute.

The results shown in Figure 4 demonstrate the superiority of DDGANI. The advantages of our method are more evident under MAR, MNAR in most cases. Specifically, under the MNAR mechanism, DDGANI achieves an ARMSE of 0.159 in Flare, notably superior to UnIMP with 0.178. This implies that our method can accurately capture the dependencies between attribute values. Particularly, our attention mechanism and data dependency plug-in can help the model to place its eyes on more distant tuples even if it cannot find clues from nearby data to impute the missing values.

E. Effectiveness of Attention Mechanism

We conduct an ablation study to demonstrate the role of the attention mechanism within DDGANI. Table V shows that removing the attention module leads to degradation of ARMSE, AMAE and accuracy of the downstream model. The drop in accuracy is particularly obvious in datasets with complex attribute correlations such as Tax. The reason is that the self-attention mechanism is employed as a preprocessing module to capture non-local dependencies inherent in tabular data. It enables DDGANI to generate globally consistent imputations and reduce spurious inconsistencies across tuples and attributes, which also indirectly facilitates subsequent FD discovery. The consistent and context-aware imputations make

downstream FD mining more stable and reliable, forming a positive feedback loop within the framework.

F. Utility of the Imputed Data

In this part of the experiment, we train a classifier with the imputed data and measure its accuracy in the test set to demonstrate the applicability of the imputed training data. Here, we fix the missing rate at 20% under the MCAR mechanism and employ Random Forest as the downstream classifier. It is worth noting that Hospital and Tax are label-free. To conduct this experiment, we take the values in the attributes EmergencyService and HasChild as labels for these two datasets, respectively, since these two attributes do not directly depend on other attributes but rather exhibit some correlations with most other attributes.

The experimental results are shown in Table IV, the first eight columns. Intuitively, the smaller the imputation error, the higher the accuracy of the downstream classifier. This has indeed been validated in many datasets. However, as mentioned in [32], many factors, not just the imputation error of the training set, affect the performance of the downstream classifier. UnIMP, for example, achieves low imputation error, this does not necessarily translate into higher downstream classification accuracy. It may smooth out class-discriminative variations that are important for classification. DDGANI outperforms other methods in all datasets. The superior performance of DDGANI is attributed to the accurate imputation and the data utility plug-in, which could keep the imputed data as close to the original distribution as possible, allowing missing value imputation to be done in a direction that helps improve the performance of the downstream classifier.

Effectiveness of Data Utility Plug-in. We conduct an ablation study to better understand the specific impact of the data utility plug-in ($\mathcal{L}_L$) on model performance. Table V shows that removal of $\mathcal{L}_L$ negatively affects the utility of the imputed data for downstream tasks. To further verify that the optimization of $\mathcal{L}_L$ can improve the estimation of data without increasing bias, we also test the performance of the classifier training with the original dataset where no missing values are introduced (denoted GroundTruth) and the dataset where tuples with missing values are removed directly (denoted Removal). We can see from Table IV that there is not much of a difference between DDGANI and GroundTruth except Tic-Tac-Toe. In

TABLE VI: Comparison with FD Involved Methods (ARMSE)

Dataset	Rate	FD	Holistic	HoloClean	Renuver	DDGANI
Adult	10%	T	0.087	<u>0.085</u>	0.109	0.084
		P	0.108	<u>0.097</u>	0.124	0.087
		R	0.087	<u>0.085</u>	0.109	0.084
	50%	T	0.506	0.485	<u>0.391</u>	0.377
		P	0.515	0.498	<u>0.405</u>	0.401
		R	0.506	0.485	<u>0.391</u>	0.377
Food	10%	T	0.524	<u>0.481</u>	0.486	0.474
		P	0.548	<u>0.500</u>	0.553	0.495
		R	0.528	<u>0.487</u>	0.516	0.483
	50%	T	0.549	<u>0.534</u>	0.559	0.528
		P	0.562	<u>0.540</u>	0.569	0.537
		R	0.555	<u>0.535</u>	0.567	0.534
Hospital	10%	T	0.082	<u>0.067</u>	0.093	0.061
		P	0.123	<u>0.116</u>	0.275	0.114
		R	0.103	<u>0.098</u>	0.102	0.091
	50%	T	0.396	<u>0.364</u>	0.408	0.343
		P	0.513	0.485	<u>0.446</u>	0.401
		R	0.456	<u>0.356</u>	0.415	0.347
Tax	10%	T	0.186	<u>0.256</u>	<u>0.168</u>	0.132
		P	<u>0.211</u>	0.286	0.246	0.179
		R	0.195	0.264	<u>0.169</u>	0.164
	50%	T	0.365	0.499	<u>0.348</u>	0.322
		P	0.414	<u>0.523</u>	<u>0.383</u>	0.355
		R	0.387	<u>0.506</u>	<u>0.353</u>	0.327

particular, we can achieve the same training results as the original Hospital dataset without missing values.

G. Effectiveness of Data Dependency Plug-in

Comparison with FD involved Methods. In this part of experiments, we introduce more baselines including (1) Holistic [8], which directly utilizes denial constraints (DCs) to guide the imputation of missing values; (2) HoloClean [30], which encodes DCs as probabilistic factors in a factor graph so that tuples violating DCs receive lower probability during inference; (3) Renuver [6] performs missing value imputation by utilizing relaxed functional dependencies (RFDs) to select candidates for missing data. The reason for selecting them as the baselines is that FDs can be represented as DCs and RFDs, and they are relatively well-established approaches for data cleaning using data constraints. We inject 10% and 50% missing values into the four datasets containing FDs under MCAR mechanisms, and compare with the above methods only regarding the attributes involved in FDs.

Table VI shows the ARMSE of imputation results. Here, “T”, “P”, and “R” denote the imputation of missing values with true FDs, potential FDs extracted from missing data, and refined FDs, respectively. We can see that DDGANI with refined FDs performs better than other methods with potential FDs, and even better than others with true FDs and refined FDs in most cases. Moreover, the superiority of DDGANI becomes even more obvious as the missing rate increases. This may be attributed to the fact that the refined FDs differ a little from the

true FDs. And the diffusion model itself could capture inter-attribute dependencies through joint distribution modeling, and attention explicitly captures attribute interactions. In addition, all evaluated methods consistently achieve better performance with refined FDs than with potential FDs, highlighting the importance of FD refinement.

We observe another interesting experimental phenomenon. DDGANI demonstrates greater robustness to noisy FDs at high missing rates (the gap between the imputation error with “T” and “R” is smaller). This may be because DDGANI integrates FDs as soft distribution-level constraints within the generative framework, allowing them to naturally weaken their influence in data-scarce scenarios. When observed data are abundant, its effect is fully unleashed. In contrast, the baselines enforce the FDs in a more rigid manner, leading to severe performance degradation at high missing rates due to noisy constraints dominating the imputation process. This phenomenon is expected, as our method adaptively balances data evidence and constraint guidance. Under high missing rates, FDs are inherently less reliable and are therefore downweighted. When data is abundant, constraints become informative and are intentionally exploited to enhance accuracy.

An Ablation Study. We also conduct an ablation study to verify the impact of data dependency plug-in ($\mathcal{L}_F$) on the model performance, which is shown in Table V. Note that the removal of $\mathcal{L}_F$ has no impact on Wine and Flare since they have no FDs. But when dealing with a dataset like Tax that contains a large number of FDs, $\mathcal{L}_F$ significantly boosts model performance. We can see from Table V that the removal of $\mathcal{L}_F$ makes ARMSE increase from 0.349 to 0.366 and AMAE increase from 0.325 to 0.345 in Tax. It confirms the efficacy of $\mathcal{L}_F$ in preserving the data dependencies of imputed data.

The Details of FD Refinement. Here, we provide the details of the FDs refinement in four datasets Adult, Food, Hospital and Tax. Table VII demonstrates the true FDs, the potential FDs mined from the tabular data with missing values, the refined FDs, and the number of refinement iterations needed. The red and gray FDs in Table VII denote the erroneous FDs and the correct FDs missing after the FDs refinement, respectively.

Table VII reveals that it is hard to avoid mining erroneous FDs from the tabular data with missing values, but the vast majority of them can be fixed correctly. We can see that we mine a total of 15 inaccurate FDs from the three datasets, of which 12 are correctly refined. Moreover, the refinement does not require too many iterations, which can prevent the erroneous FDs from having a continuous negative effect on the missing value imputation. However, there are three erroneous FDs that remain after the phase of refinement in Tax. This may be due to the strong correlation between the attributes in these FDs (e.g., State→Areacode) or the values of attributes on the left-hand side being too discrete (e.g., SingleExemp, MarriedExemp→Areacode). These three FDs hold true in most tuples, making it difficult to learn to impute the missing values that do not follow such patterns. It can also be seen

TABLE VII: Details of FDs Refinement

Dataset	True FDs	Potential FDs	Refined FDs	# Iterations
Adult	education-num→education education→education-num	education-num→education education→education-num marital-status, sex→relationship	education-num→education education→education-num	1
Food	DBA_Name, Address→Facility_Type DBA_Name, Zip→Facility_Type DBA_Name, Zip→Risk DBA_Name, Address→Risk Address→Zip	DBA_Name, Address→Facility_Type DBA_Name, Zip→Facility_Type DBA_Name, Risk→Facility_Type DBA_Name, Inspection_Type→Facility_Type Address, Risk→Facility_Type DBA_Name, Address→Risk DBA_Name, Inspection_Type→Risk DBA_Name, Zip→Risk Address→Zip	DBA_Name, Zip→Facility_Type DBA_Name, Address→Facility_Type DBA_Name, Address→Risk DBA_Name, Inspection_Type→Risk DBA_Name, Zip→Risk Address→Zip	6
Hospital	HospitalName→City City→State HospitalName→State HospitalName→HospitalOwner MeasureCode→Condition MeasureName→MeasureCode MeasureCode→MeasureName	HospitalName→City MeasureCode→State MeasureName→State City→State HospitalName→State Condition, HospitalOwner→State HospitalName→HospitalOwner City, MeasureCode→HospitalOwner City, MeasureName→HospitalOwner MeasureCode→Condition MeasureName→Condition MeasureName→MeasureCode MeasureCode→MeasureName	HospitalName→City (missing) City→State HospitalName→State HospitalName→HospitalOwner MeasureCode→Condition MeasureName→Condition MeasureName→MeasureCode MeasureCode→MeasureName	4
Tax	Areacode→State State, MaritalStatus→SingleExemp State, MarriedExemp→SingleExemp Areacode, MaritalStatus→SingleExemp Areacode, MarriedExemp→SingleExemp State, MaritalStatus→MarriedExemp State, SingleExemp→MarriedExemp Areacode, MaritalStatus→MarriedExemp Areacode, SingleExemp→MarriedExemp	State→Areacode SingleExemp, MarriedExemp→Areacode Areacode→State City, MarriedExemp→State SingleExemp, MarriedExemp→State Areacode→SingleExemp State→SingleExemp City, MaritalStatus→SingleExemp State, MaritalStatus→MarriedExemp State, SingleExemp→MarriedExemp Areacode, MaritalStatus→MarriedExemp Areacode, SingleExemp→MarriedExemp City, MaritalStatus→MarriedExemp City, SingleExemp→MarriedExemp	State→Areacode SingleExemp, MarriedExemp→Areacode Areacode→State SingleExemp, MarriedExemp→State Areacode, MaritalStatus→SingleExemp Areacode, MarriedExemp→SingleExemp State, MaritalStatus→SingleExemp State, MarriedExemp→SingleExemp State, MaritalStatus→MarriedExemp State, SingleExemp→MarriedExemp Areacode, MaritalStatus→MarriedExemp Areacode, SingleExemp→MarriedExemp	2

that a correct FD in Hospital is mined in the beginning but mistakenly discarded in the refinement phase. The reason is that the imputation is not accurate enough, which makes the confidence of the tuple violating this FD slightly higher than the confidence of FD.

H. Impact of Hyperparameters

We vary the diffusion timestep T from 1 to 12 and the threshold κ in the attention mechanism from 0.05 to 1 to show their impact on the performance of missing value imputation. The results are shown in Figure 5.

As we can see from Figure 5(a), the imputation error decreases dramatically as T grows (The ARMSE decreases from 0.220 at $T = 1$ to 0.195 at $T = 6$ in Adult) but remains essentially stable when T exceeds a certain value, *e.g.*, $T = 6$. The results confirm the effectiveness of decomposing the generation process from a one-shot manner into multiple diffusion steps and the role of denoising diffusion GANs in shortening the diffusion process. As for the attention threshold κ , we observe a decline in imputation performance with increasing κ , which suggests that a smaller value of κ allows the model to focus more on the data associated with the missing values, effectively preventing decision biases caused by data imbalances. Due to the presence of the Learner, the accuracy

TABLE VIII: Runtime and Resource Consumption

Method	Time (s)		CPU (%)		GPU (%)		GPU Mem. (GB)	
	AVG	Avg Rank	MAX	Avg Rank	MAX	Avg Rank	MAX	Avg Rank
MissFI	35	2.0	73.60	3.73	0.00	1.0	0.35	1.0
AimNet	702	4.5	95.00	5.64	5.30	2.0	0.75	2.0
TabImpute	32	1.5	20.70	2.00	100.00	8.00	18.60	6.55
PMAE	74	3.5	19.50	1.18	10.00	3.0	0.92	3.0
UnIMP	12463	7.3	58.50	5.45	100.00	7.09	47.98	7.82
DiffPuter	666	6.4	84.00	7.82	74.00	5.36	4.78	5.09
TabDiff	559	6.1	70.00	5.91	100.00	6.55	18.78	6.27
DDGANI	245	4.5	64.70	4.45	13.00	4.18	2.60	4.27

of the downstream classifier varies within a certain controlled variance, less affected by these two hyperparameters.

I. Runtime and Resource Consumption Comparison

In the end, we conduct experiments to compare the runtime and resource consumption of different methods. Table VIII demonstrate the average imputation time (AVG), the peak resource consumption (MAX) and average ranking (Avg Rank) of these methods across all datasets under the corresponding metric. Detailed experimental results can be found in the appendix. Note that the imputation time corresponds to end-to-end runtime, measured from the input of tabular data until the completion of missing value imputation. The average ranking

is computed by ranking methods per dataset and averaging over all datasets. The average captures the absolute average performance, whereas the average ranking mitigates the impact of dataset-dependent scale differences by comparing methods on a per-dataset basis and averaging their relative rankings.

The experimental results reveal that DDGANI exhibits an intermediate level of imputation time and resource usage, underperforming lightweight machine learning based methods (e.g., MissFI) and one-shot generative methods such as PMAE but outperforming the vast majority of other approaches. The denoising diffusion framework adopted in DDGANI significantly reduces the number of diffusion steps. It approaches the target distribution step by step, with each step requiring only small distributional shifts. This makes the task relatively simple and stable, resulting in faster convergence than other methods. DDGANI is less efficient than the lightweight methods such as PMAE but also has lower imputation error. The trade-off between accuracy and efficiency is important in scenarios where data quality directly affects downstream tasks, such as data analytics or model training. In these settings, even moderate improvements in imputation accuracy can lead to measurable downstream benefits, while efficiency remains a secondary concern. It should be noted that TabImpute utilizes a pre-trained model without fine-tuning. This explains why it ranks so highly in terms of runtime while consuming significant resources.

As for the resource consumption, we note that DDGANI exhibits relatively high peak CPU utilization, but this is expected given its design. Our method integrates attention mechanism, constraint-aware imputation, and adversarial supervision, which introduce additional CPU-side orchestration and control logic. Peak CPU utilization indicates short-lived bursts of intensive CPU activity, while relatively low average utilization shows that such bursts are infrequent and do not dominate the overall execution time. In terms of GPU utilization, DiffPuter, which also employs the diffusion model, exhibits lower average GPU utilization than DDGANI and consumes less memory. This is because of the lighter diffusion architecture with limited per-step computation. However, this also means that it requires more iteration steps, thereby increasing the runtime. DDGANI intentionally introduces additional modules, significantly increasing the per-step computation and more fully utilizing GPU resources, which directly translates into improved accuracy and consistency of the imputation rather than unnecessary overhead.

VIII. RELATED WORK

Statistical Imputation Approaches. Statistical-based methods typically calculate statistics such as the mean, median, and mode of the observed data to estimate missing values. Several statistical methods can also be combined to impute missing data [7], [38]. However, most of these methods only work well for imputing numerical data, which is not suitable for some real-world scenarios and is far from high quality.

Heuristic Based Imputation Approaches. Heuristic methods have been proposed to effectively estimate the missing values

without violating the integrity of the original data [6], [8], [29], [34]. There exist some methods directly mining constraints from missing data for the subsequent imputation [3], [22] and estimating the confidence of discovered constraints based on the tuple frequency while ignoring each tuple’s confidence. Graf [29] iteratively repairs dirty data and inaccurate CFDs based on their confidence, but requires a great deal of complete tuples to accomplish, which is impractical when dealing with a large number of missing values.

Machine Learning Based Imputation Approaches. ML-based imputation methods generally train ML models to predict missing values [24], [31], [35], [37]. Although these ML-based imputation methods outperform statistical ones [26], they may have strict distributional assumptions or struggle with categorical data.

Deep Learning Based Imputation Approaches. DL-based methods generally outperform the aforementioned approaches as they can capture the correlations between data through multiple layers of nonlinear computations, enabling effective imputation of missing data. Some studies such as DataWig [4] train a DL model for each attribute. However, if they impute one attribute incorrectly, it can greatly affect the accuracy of the imputation on other attributes. Other approaches, especially those based on generative models, can reconstruct complete data at once. TabCSDI [45], DiffPuter [44] and TabDiff [33] all employ diffusion models to impute incomplete tabular data, and the experimental results show that our method outperforms the above baselines. Pre-trained models have also been used for estimating the missing values such as TabImpute [12], TURL [10], RPT [36] and UnIMP [39] leverage pre-trained language models even large language models.

IX. CONCLUSION

We have proposed DDGANI, a missing value imputation framework based on denoising diffusion GAN. We have incorporated the self-attention mechanism, which enables the generative model to capture more relevant contextual information. We have designed two plugins that can be chosen based on data characteristics and downstream tasks, which ensure the utility of imputed data and the preservation of inherent data dependencies. Extensive experimental results have demonstrated DDGANI outperforms state-of-the-art approaches.

ACKNOWLEDGEMENT

This work was supported by the National Natural Science Foundation of China (62372034, 61902017) and CCF-Huawei Populus Grove Fund.

REFERENCES

- [1] N. S. Altman. An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician*, 46(3):175–185, 1992.
- [2] O. F. Ayilara, L. Zhang, T. T. Sajobi, R. Sawatzky, E. Bohm, and L. M. Lix. Impact of missing data on bias and precision when estimating change in patient-reported outcomes from a clinical registry. *Health and quality of life outcomes*, 17:1–9, 2019.
- [3] L. Berti-Équille, H. Harmouch, F. Naumann, N. Novelli, and S. Thirumuruganathan. Discovery of genuine functional dependencies from relational data with missing values. *Proceedings of the VLDB Endowment*, 11(8):880–892, 2018.

- [4] F. Biessmann, T. Rukat, P. Schmidt, P. Naidu, S. Schelter, A. Taptunov, D. Lange, and D. Salinas. Datawig: Missing value imputation for tables. *Journal of Machine Learning Research*, 20(175):1–6, 2019.
- [5] P. Bohannon, W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis. Conditional functional dependencies for data cleaning. In *2007 IEEE 23rd international conference on data engineering*, pages 746–755. IEEE, 2006.
- [6] B. Breve, L. Caruccio, V. Deufemia, and G. Polese. Renuver: A missing value imputation algorithm based on relaxed functional dependencies. In *EDBT*, pages 1–52, 2022.
- [7] S. V. Buuren, J. P. Brand, C. Groothuis-Oudshoorn, and D. B. Rubin. Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, 76(12):1049–1064, 2006.
- [8] X. Chu, I. F. Ilyas, and P. Papotti. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pages 458–469. IEEE, 2013.
- [9] H. Cramér. *Mathematical methods of statistics*, volume 43. Princeton university press, 1999.
- [10] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu. Turl: Table understanding through representation learning. *ACM SIGMOD Record*, 51(1):33–40, 2022.
- [11] A. Farhangfar, L. A. Kurgan, and W. Pedrycz. A novel framework for imputation of missing values in databases. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 37(5):692–709, 2007.
- [12] J. Feitelberg, D. Saha, K. Choi, Z. Ahmad, A. Agarwal, and R. Dwivedi. Tabimpute: Accurate and fast zero-shot missing-data imputation with a pre-trained transformer. *arXiv preprint arXiv:2510.02625*, 2025.
- [13] H. Hammad Alharbi and M. Kimura. Missing data imputation using data generated by gan. In *2020 the 3rd International Conference on Computing and Big Data*, pages 73–77, 2020.
- [14] X. Hong and S. Hao. Imputation of missing values in training data using variational autoencoder. In *2023 IEEE 39th International Conference on Data Engineering Workshops (ICDEW)*, pages 49–54. IEEE, 2023.
- [15] Y. Huhtala, J. Kärkkäinen, P. Porkka, and H. Toivonen. Tane: An efficient algorithm for discovering functional and approximate dependencies. *The computer journal*, 42(2):100–111, 1999.
- [16] N. B. Ipsen, P.-A. Mattei, and J. Frellsen. not-miwa: Deep generative modelling with missing not at random data. In *ICLR 2021-International Conference on Learning Representations*, 2021.
- [17] D. Jarrett, B. C. Ceber, T. Liu, A. Curth, and M. van der Schaar. Hyperimpute: Generalized iterative imputation with automatic model selection. In *International Conference on Machine Learning*, pages 9916–9937. PMLR, 2022.
- [18] S. R. Jeffery, M. Garofalakis, and M. J. Franklin. Adaptive cleaning for rfid data streams. In *VLDB*, volume 6, pages 163–174. Citeseer, 2006.
- [19] J. Josse, J. Pagès, and F. Husson. Multiple imputation in principal component analysis. *Advances in data analysis and classification*, 5:231–246, 2011.
- [20] J. Kim, K. Lee, and T. Park. To predict or not to predict? proportionally masked autoencoders for tabular data imputation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 17886–17894, 2025.
- [21] S. C.-X. Li, B. Jiang, and B. Marlin. Misgan: Learning from incomplete data with generative adversarial networks. In *International Conference on Learning Representations*, 2018.
- [22] E. Livshits, A. Heidari, I. F. Ilyas, and B. Kimelfeld. Approximate denial constraints. *arXiv preprint arXiv:2005.08540*, 2020.
- [23] M. Mahdavi, Z. Abedjan, R. Castro Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang. Raha: A configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data*, pages 865–882, 2019.
- [24] R. Mazumder, T. Hastie, and R. Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *The Journal of Machine Learning Research*, 11:2287–2322, 2010.
- [25] J. T. McCoy, S. Kroon, and L. Auret. Variational autoencoders for missing data imputation with application to a simulated milling circuit. *IFAC-PapersOnLine*, 51(21):141–146, 2018.
- [26] X. Miao, Y. Wu, L. Chen, Y. Gao, J. Wang, and J. Yin. Efficient and effective data imputation with influence functions. *Proceedings of the VLDB Endowment*, 15(3):624–632, 2021.
- [27] R. Mitra, S. F. McGough, T. Chakraborti, C. Holmes, R. Copping, N. Hagenbuch, S. Biedermann, J. Noonan, B. Lehmann, A. Shenvi, et al. Learning from data with structured missingness. *Nature Machine Intelligence*, 5(1):13–23, 2023.
- [28] A. Nazabal, P. M. Olmos, Z. Ghahramani, and I. Valera. Handling incomplete heterogeneous data using vaes. *Pattern Recognition*, 107:107501, 2020.
- [29] J. Peng, D. Shen, N. Tang, T. Liu, Y. Kou, T. Nie, H. Cui, and G. Yu. Self-supervised and interpretable data cleaning with sequence generative adversarial networks. *Proceedings of the VLDB Endowment*, 16(3):433–446, 2022.
- [30] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré. Holoclean: Holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment*, 10(11), 2017.
- [31] P. Royston and I. R. White. Multiple imputation by chained equations (mice): implementation in stata. *Journal of statistical software*, 45:1–20, 2011.
- [32] T. Shadbahr, M. Roberts, J. Stanczuk, J. Gilbey, P. Teare, S. Dittmer, M. Thorpe, R. V. Torné, E. Sala, P. Lió, et al. The impact of imputation quality on machine learning classifiers for datasets with missing values. *Communications Medicine*, 3(1):139, 2023.
- [33] J. Shi, M. Xu, H. Hua, H. Zhang, S. Ermon, and J. Leskovec. Tabdiff: a mixed-type diffusion model for tabular data generation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [34] S. Song, Y. Sun, A. Zhang, L. Chen, and J. Wang. Enriching data imputation under similarity rule constraints. *IEEE transactions on knowledge and data engineering*, 32(2):275–287, 2018.
- [35] D. J. Stekhoven and P. Bühlmann. Missforest—non-parametric missing value imputation for mixed-type data. *Bioinformatics*, 28(1):112–118, 2012.
- [36] N. Tang, J. Fan, F. Li, J. Tu, X. Du, G. Li, S. Madden, and M. Ouzzani. RPT: relational pre-trained transformer is almost all you need towards democratizing data preparation. *Proceedings of the VLDB Endowment*, 14(8):1254–1261, 2021.
- [37] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman. Missing value estimation methods for dna microarrays. *Bioinformatics*, 17(6):520–525, 2001.
- [38] S. van Buuren. Multiple imputation of discrete and continuous data by fully conditional specification. *Statistical Methods in Medical Research*, 16(3):219–242, 2007.
- [39] J. Wang, K. Wang, Y. Zhang, W. Zhang, X. Xu, and X. Lin. On llm-enhanced mixed-type data imputation with high-order message passing. *Proceedings of the VLDB Endowment*, 18(10):3421–3434, Sept. 2025.
- [40] Y. Wen, Y. Wang, K. Yi, J. Ke, and Y. Shen. Diffimpute: Tabular data imputation with denoising diffusion probabilistic model. In *2024 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6. IEEE, 2024.
- [41] R. Wu, A. Zhang, I. Ilyas, and T. Rekatsinas. Attention-based learning for missing data imputation in holoclean. *Proceedings of Machine Learning and Systems*, 2:307–325, 2020.
- [42] Z. Xiao, K. Kreis, and A. Vahdat. Tackling the generative learning trilemma with denoising diffusion gans. *arXiv preprint arXiv:2112.07804*, 2021.
- [43] J. Yoon, J. Jordon, and M. Schaar. Gain: Missing data imputation using generative adversarial nets. In *International Conference on Machine Learning*, pages 5689–5698. PMLR, 2018.
- [44] H. Zhang, L. Fang, Q. Wu, and P. S. Yu. Diffputer: Empowering diffusion models for missing data imputation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [45] S. Zheng and N. Charoenphakdee. Diffusion models for missing value imputation in tabular data. *arXiv preprint arXiv:2210.17128*, 2022.

APPENDIX

A. Additional Experimental Results on Baselines

In this appendix, we introduce additional baselines, which are representative of missing value imputation methods of various types, including (1) Mean [11], which imputes the missing values by the mean of each numerical attribute and the mode of each categorical attribute; (2) HoloClean [30], which is a probabilistic factor-graph-based data cleaning framework jointly modeling integrity constraints, statistical correlations

and external signals; (3) GAIN [43], which employs a GAN-based model and utilizes a hinting mechanism to optimize the model for imputing missing values; (4) Graf [29], which uses sequence GANs to learn conditional functional dependencies from the dirty data and imputes the missing values based on these learned CFDs; (5) Renuver [6], which recovers missing values via dependency-driven candidate generation and statistical validation, and (6) TabCSDI, which adopts the conditional score-based diffusion model that extends time-series diffusion techniques to tabular missing value imputation. Therefore, the appendix contains detailed experimental results for 14 baselines, including overall evaluation, comparison under different missing rates and types, accuracy of downstream classifier, details of imputation time and resource consumption comparison. The experimental results could further demonstrate the superiority of DDGANI.

B. The Extensions to Support CFDs

We could extend the data dependency plugin to support CFDs by covering three aspects: mining, modeling, and refinement.

CFD Mining. A CFD is required to hold only on a conditional subset of the table, rather than on the entire dataset. Consequently, the key difference from FD mining lies in the validity domain: FD is enforced globally, whereas CFD only holds within pattern-specified subtables. Specifically, for each candidate FD $\varphi : Z \rightarrow A$, we enumerate the pattern T_p on $Z \cup \{A\}$ consisting entirely of wildcards to the patterns consisting entirely of specific attribute values, verifying whether the FD holds for the subset of tuples specified by the pattern. These tuples must satisfy the condition of containing no missing values in $Z \cup \{A\}$, following the same criteria as FD mining. Additionally, the support of this FD in the pattern-specific subset must exceed a specified threshold. Note that if φ holds for a more generic pattern T_p , we skip the verification for the patterns covered within T_p .

CFD Modeling. We also employ the FD model proposed in our paper for CFD modeling. Concretely, we cluster the extracted CFDs, placing those that embed the same FD into a single group. Then, we construct an FD model for a set of CFDs and maintain a pattern collection. If a tuple matches any pattern in this collection, we expect its attribute values of A to match the output of the FD model.

CFD Refinement. Given a CFD $\phi : (Z \rightarrow A, T_p)$ mined from incomplete data, its confidence within a pattern-specific subtable is calculated in the same manner as FD confidence. We will refine a CFD ϕ if there exists a high-confidence tuple (*i.e.*, greater than the confidence of ϕ) that violates it. Note that in the CFD setting, inaccuracies may arise not only from the dependency but also from inappropriate conditional domains. Therefore, we first perform pattern specialization, which involves changing wildcard conditions into concrete values to narrow the conditional domain while keeping the embedded FD unchanged. If narrowing the scope of the pattern does not work, it indicates that the left-hand side of embedded FD may lack sufficient discriminative power. Thus, we add

TABLE IX: Overall Evaluation of all Methods (Average $\pm$ Std)

Dataset	Metrics	Mean	MissFI	HoloClean	GAIN	AimNet	Graf	Renuver	TabImpute	PMAE	UnIMP	TabCSDI	DiffPuter	TabDiff	DDGANI
Wine	ARMSE	0.296 $\pm$ 0.000	0.151 $\pm$ 0.003	0.244 $\pm$ 0.009	0.171 $\pm$ 0.007	0.241 $\pm$ 0.014	0.297 $\pm$ 0.014	0.207 $\pm$ 0.001	0.153 $\pm$ 0.001	0.157 $\pm$ 0.004	0.132$\pm$0.003	0.211 $\pm$ 0.012	0.179 $\pm$ 0.012	0.154 $\pm$ 0.006	0.145 $\pm$ 0.001
	AMAE	0.250 $\pm$ 0.000	0.113 $\pm$ 0.002	0.188 $\pm$ 0.007	0.135 $\pm$ 0.007	0.185 $\pm$ 0.010	0.251 $\pm$ 0.012	0.172 $\pm$ 0.001	0.117 $\pm$ 0.001	0.121 $\pm$ 0.004	0.110 $\pm$ 0.003	0.131 $\pm$ 0.011	0.126 $\pm$ 0.013	0.116 $\pm$ 0.009	0.106$\pm$0.001
Spam	ARMSE	0.053 $\pm$ 0.000	0.048 $\pm$ 0.000	0.051 $\pm$ 0.001	0.051 $\pm$ 0.001	0.051 $\pm$ 0.007	0.054 $\pm$ 0.002	0.050 $\pm$ 0.010	0.054 $\pm$ 0.004	0.050 $\pm$ 0.000	0.048 $\pm$ 0.002	0.051 $\pm$ 0.012	0.050 $\pm$ 0.005	0.049 $\pm$ 0.002	0.047$\pm$0.000
	AMAE	0.022 $\pm$ 0.000	0.023 $\pm$ 0.000	0.023 $\pm$ 0.002	0.022 $\pm$ 0.000	0.022 $\pm$ 0.001	0.022 $\pm$ 0.000	0.022 $\pm$ 0.010	0.021 $\pm$ 0.003	0.021$\pm$0.000	0.022 $\pm$ 0.002	0.022 $\pm$ 0.015	0.021 $\pm$ 0.002	0.021 $\pm$ 0.001	0.021$\pm$0.000
Tic-Tac-Toe	ARMSE	0.590 $\pm$ 0.009	0.529 $\pm$ 0.008	0.567 $\pm$ 0.006	0.537 $\pm$ 0.013	0.525 $\pm$ 0.007	0.523 $\pm$ 0.006	0.533 $\pm$ 0.003	0.533 $\pm$ 0.004	0.521 $\pm$ 0.004	0.518 $\pm$ 0.005	0.530 $\pm$ 0.000	0.526 $\pm$ 0.002	0.516 $\pm$ 0.012	0.512$\pm$0.002
	AMAE	0.590 $\pm$ 0.009	0.529 $\pm$ 0.008	0.567 $\pm$ 0.006	0.537 $\pm$ 0.013	0.525 $\pm$ 0.007	0.523 $\pm$ 0.006	0.533 $\pm$ 0.003	0.533 $\pm$ 0.004	0.521 $\pm$ 0.004	0.518 $\pm$ 0.005	0.530 $\pm$ 0.000	0.526 $\pm$ 0.002	0.516 $\pm$ 0.012	0.512$\pm$0.002
Flare	ARMSE	0.260 $\pm$ 0.000	0.240 $\pm$ 0.000	0.244 $\pm$ 0.005	0.200 $\pm$ 0.009	0.241 $\pm$ 0.002	0.262 $\pm$ 0.004	0.258 $\pm$ 0.003	0.257 $\pm$ 0.003	0.206 $\pm$ 0.005	0.186 $\pm$ 0.004	0.221 $\pm$ 0.000	0.215 $\pm$ 0.007	0.214 $\pm$ 0.003	0.173$\pm$0.003
	AMAE	0.260 $\pm$ 0.000	0.240 $\pm$ 0.000	0.244 $\pm$ 0.005	0.200 $\pm$ 0.009	0.241 $\pm$ 0.002	0.262 $\pm$ 0.004	0.258 $\pm$ 0.003	0.257 $\pm$ 0.003	0.206 $\pm$ 0.005	0.186 $\pm$ 0.004	0.221 $\pm$ 0.000	0.215 $\pm$ 0.007	0.214 $\pm$ 0.003	0.173$\pm$0.003
Adult	ARMSE	0.341 $\pm$ 0.000	0.258 $\pm$ 0.000	0.298 $\pm$ 0.006	0.250 $\pm$ 0.007	0.293 $\pm$ 0.001	0.338 $\pm$ 0.000	0.238 $\pm$ 0.010	0.236 $\pm$ 0.006	0.228 $\pm$ 0.011	0.196 $\pm$ 0.008	0.301 $\pm$ 0.002	0.261 $\pm$ 0.013	0.218 $\pm$ 0.021	0.195$\pm$0.000
	AMAE	0.325 $\pm$ 0.000	0.244 $\pm$ 0.000	0.281 $\pm$ 0.004	0.234 $\pm$ 0.008	0.272 $\pm$ 0.001	0.323 $\pm$ 0.000	0.238 $\pm$ 0.010	0.236 $\pm$ 0.006	0.228 $\pm$ 0.011	0.196 $\pm$ 0.008	0.301 $\pm$ 0.002	0.261 $\pm$ 0.013	0.218 $\pm$ 0.021	0.195$\pm$0.000
Food	ARMSE	0.752 $\pm$ 0.012	0.689 $\pm$ 0.013	0.639 $\pm$ 0.011	0.668 $\pm$ 0.013	0.632 $\pm$ 0.013	0.639 $\pm$ 0.013	0.646 $\pm$ 0.013	0.626 $\pm$ 0.005	0.617 $\pm$ 0.008	0.596 $\pm$ 0.011	0.632 $\pm$ 0.013	0.608 $\pm$ 0.008	0.599 $\pm$ 0.013	0.589$\pm$0.004
	AMAE	0.721 $\pm$ 0.008	0.683 $\pm$ 0.010	0.631 $\pm$ 0.011	0.652 $\pm$ 0.012	0.625 $\pm$ 0.013	0.635 $\pm$ 0.013	0.638 $\pm$ 0.013	0.623 $\pm$ 0.005	0.613 $\pm$ 0.006	0.590 $\pm$ 0.011	0.629 $\pm$ 0.012	0.604 $\pm$ 0.008	0.592 $\pm$ 0.013	0.584$\pm$0.004
Hospital	ARMSE	0.618 $\pm$ 0.000	0.254 $\pm$ 0.003	0.173 $\pm$ 0.009	0.365 $\pm$ 0.026	0.169 $\pm$ 0.001	0.267 $\pm$ 0.003	0.255 $\pm$ 0.003	0.294 $\pm$ 0.001	0.246 $\pm$ 0.004	0.142 $\pm$ 0.005	0.320 $\pm$ 0.005	0.233 $\pm$ 0.003	0.205 $\pm$ 0.020	0.133$\pm$0.002
	AMAE	0.610 $\pm$ 0.000	0.239 $\pm$ 0.003	0.145 $\pm$ 0.008	0.351 $\pm$ 0.026	0.139 $\pm$ 0.001	0.258 $\pm$ 0.003	0.255 $\pm$ 0.003	0.281 $\pm$ 0.001	0.231 $\pm$ 0.004	0.125 $\pm$ 0.006	0.260 $\pm$ 0.005	0.224 $\pm$ 0.003	0.196 $\pm$ 0.021	0.115$\pm$0.002
Tax	ARMSE	0.615 $\pm$ 0.000	0.380 $\pm$ 0.011	0.432 $\pm$ 0.026	0.558 $\pm$ 0.026	0.422 $\pm$ 0.026	0.369 $\pm$ 0.082	0.325 $\pm$ 0.005	0.283 $\pm$ 0.040	0.299 $\pm$ 0.030	0.266 $\pm$ 0.002	0.325 $\pm$ 0.060	0.303 $\pm$ 0.002	0.278 $\pm$ 0.009	0.234$\pm$0.000
	AMAE	0.600 $\pm$ 0.000	0.360 $\pm$ 0.009	0.396 $\pm$ 0.034	0.526 $\pm$ 0.039	0.391 $\pm$ 0.026	0.355 $\pm$ 0.076	0.310 $\pm$ 0.003	0.262 $\pm$ 0.037	0.279 $\pm$ 0.030	0.234 $\pm$ 0.003	0.312 $\pm$ 0.060	0.286 $\pm$ 0.002	0.244 $\pm$ 0.010	0.212$\pm$0.000
Contraceptive	ARMSE	0.351 $\pm$ 0.003	0.328 $\pm$ 0.003	0.335 $\pm$ 0.006	0.341 $\pm$ 0.006	0.337 $\pm$ 0.003	0.313 $\pm$ 0.004	0.338 $\pm$ 0.008	0.305 $\pm$ 0.009	0.324 $\pm$ 0.005	0.314 $\pm$ 0.015	0.312 $\pm$ 0.001	0.308 $\pm$ 0.007	0.308 $\pm$ 0.014	0.290$\pm$0.001
	AMAE	0.339 $\pm$ 0.003	0.318 $\pm$ 0.002	0.327 $\pm$ 0.006	0.331 $\pm$ 0.006	0.328 $\pm$ 0.003	0.302 $\pm$ 0.004	0.331 $\pm$ 0.008	0.296 $\pm$ 0.008	0.315 $\pm$ 0.005	0.302 $\pm$ 0.011	0.301 $\pm$ 0.001	0.293 $\pm$ 0.008	0.295 $\pm$ 0.014	0.281$\pm$0.001
Australian	ARMSE	0.295 $\pm$ 0.004	0.269 $\pm$ 0.013	0.270 $\pm$ 0.006	0.264 $\pm$ 0.007	0.255 $\pm$ 0.006	0.265 $\pm$ 0.008	0.281 $\pm$ 0.004	0.260 $\pm$ 0.005	0.266 $\pm$ 0.003	0.257 $\pm$ 0.006	0.271 $\pm$ 0.001	0.245 $\pm$ 0.006	0.267 $\pm$ 0.008	0.237$\pm$0.003
	AMAE	0.273 $\pm$ 0.004	0.254 $\pm$ 0.014	0.245 $\pm$ 0.007	0.247 $\pm$ 0.008	0.232 $\pm$ 0.004	0.247 $\pm$ 0.005	0.264 $\pm$ 0.002	0.238 $\pm$ 0.005	0.248 $\pm$ 0.003	0.239 $\pm$ 0.006	0.247 $\pm$ 0.001	0.232 $\pm$ 0.007	0.244 $\pm$ 0.007	0.226$\pm$0.002

TABLE X: Accuracy of Downstream Classifier (All Baselines)

Dataset	Mean	MissFI	HoloClean	GAIN	AimNet	Graf	Remover	TabImpute	PMAE	UnIMP	TabCSDI	DiffPutter	TabDiff	DDGANI	Removal	GroundTruth
Wine	94.44±0.00	99.44±1.11	93.33±0.02	97.78±1.11	93.52±1.31	95.00±3.24	98.89±0.03	99.44±0.01	99.54±1.11	99.76±0.02	94.44±0.00	96.67±0.09	98.45±1.24	99.78±0.12	88.89±0.00	100.00±0.00
Spam	93.38±0.00	93.25±0.00	92.20±0.01	93.05±0.08	92.67±2.36	93.21±0.63	94.05±0.03	94.08±0.03	94.02±0.05	94.26±0.00	93.70±0.02	94.21±0.22	93.90±0.03	94.81±0.03	-	95.55±0.00
Tic-Tac-Toe	80.21±0.00	83.44±0.88	83.02±0.01	86.98±1.00	83.68±0.49	86.67±0.63	86.20±0.00	83.65±0.00	87.63±0.88	88.31±0.15	85.15±0.00	87.60±0.04	88.36±0.72	88.72±1.30	77.92±0.00	98.85±0.00
Flare	78.60±0.00	80.06±0.79	79.91±0.01	79.91±1.34	77.57±1.01	80.22±1.34	78.97±0.01	79.81±0.01	79.57±0.80	80.30±1.25	78.17±0.02	79.50±0.05	79.41±0.62	80.37±1.10	78.60±0.00	81.79±0.00
Adult	84.92±0.00	85.00±0.05	82.40±0.02	84.80±0.27	82.62±0.43	84.21±0.32	80.00±0.03	83.35±0.02	84.96±0.08	84.98±0.04	83.70±0.01	84.85±0.04	85.04±0.03	85.12±0.09	81.43±0.00	85.34±0.00
Food	67.20±0.00	76.00±0.20	77.35±0.10	76.00±0.03	76.86±0.03	76.50±0.09	76.75±0.05	75.50±0.08	77.25±0.09	77.60±0.03	76.20±0.08	78.00±0.13	78.00±0.05	78.98±0.11	76.10±0.00	82.02±0.00
Hospital	96.50±0.00	96.50±0.00	96.80±0.00	96.60±0.20	96.70±0.24	96.70±0.24	96.25±0.00	96.50±0.00	96.90±0.00	96.81±0.14	96.62±0.00	96.60±0.02	96.80±0.02	97.00±0.00	95.00±0.00	97.00±0.00
Tax	86.48±0.00	87.04±0.82	85.75±0.07	85.75±0.56	84.91±0.37	86.77±0.61	63.75±0.06	86.87±0.07	87.65±0.45	87.89±0.46	87.03±0.26	87.84±0.45	87.32±0.36	88.73±0.66	86.29±0.00	89.41±0.00
Contraceptive	53.15±0.00	52.09±0.32	51.12±0.02	51.53±0.61	49.27±1.76	49.22±0.54	51.86±0.00	52.68±0.00	54.64±0.45	55.02±0.22	52.54±0.01	53.10±0.04	54.46±0.32	55.59±0.36	49.56±0.00	55.98±0.00
Australian	86.26±0.00	86.71±0.34	86.23±0.02	87.25±1.18	87.20±0.49	84.06±0.65	86.96±0.00	87.83±0.00	87.83±0.12	87.98±0.15	87.25±0.01	85.54±0.05	88.01±0.22	88.13±0.29	82.90±0.00	89.86±0.00

new attributes to the left-hand side of the embedded FD (following the same strategy as FD refinement proposed in our original paper) and search for suitable patterns from general to specific. If the number of attributes in the left-hand side after appending exceeds the threshold τ , we discard this CFD when it still does not hold. We do not consider removing attributes from the left-hand side of the embedded FD or enlarging the pattern domain, since these relaxing operations may introduce additional violations and cannot make the CFD valid. Another scenario arises where, as imputation progresses, the support of a candidate CFD exceeds the threshold. This requires us to monitor those candidate CFDs approaching the threshold during mining and determine whether they should be added to the set of final potential CFDs.

We test the above methods on the Food dataset, and the details of its CFD refinement could be found in Table XV. Due to the large number of mined CFDs, Table XV only presents a subset of representative results.

C. Hyperparameter Configuration Details

Loss weights α and β . In addition to minimizing the reconstruction loss $\mathcal{L}_R$, the generator in DDGANI is optimized jointly by adversarial loss $\mathcal{L}_M$ and dependency loss $\mathcal{L}_F$. The primary role of α and β is not to impose an explicit preference toward either loss term, but rather to ensure that their magnitudes remain consistent. Thus, in practice, we could monitor the magnitudes of $\mathcal{L}_R$, $\mathcal{L}_M$ and $\mathcal{L}_F$ during the early training stage and choose α and β such that these losses contribute comparable gradients during backpropagation. This empirical strategy effectively prevents one loss from dominating or being neglected in joint optimization.

The threshold τ . The threshold τ limits the maximum number of attributes in the LHS of mined FDs. Its setting is based on the empirical observation that FD with more than three attributes on the left-hand side is extremely rare. In addition, as mentioned in our paper, if it is set too large, the FD will hold mostly because there are no two tuples with the same value in attributes on the left-hand side, which does not work for reasoning about missing values. We evaluate the impact of varying τ from 1 to 5 on missing value imputation using all FD-containing datasets and the experimental results can be found in Figure 8. Since the left-hand side of FDs holds for no more than two attributes in these datasets, increasing τ from 1 to 2 generally leads to performance improvements, but further increases have negligible impact. This is because even if some FDs with more left-hand-side attributes are discovered, they have little effect on imputation or are quickly corrected. Therefore, for a new dataset, τ can be set to 3, 4, or 5, since it is generally not considered FDs with so many attributes on the left side.

Other training parameters. As for other training parameters of deep learning models, such as learning rate and number of epochs, or parameters introduced by the diffusion model, like the number of diffusion steps, our method follows standard configurations widely adopted in prior work.

TABLE XI: Imputation Time Evaluation (in seconds)
(Avg Rank is computed by ranking methods per dataset and averaging over all datasets)

Method	Wine	Spam	Tic.	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	AVG	Avg Rank ↓
Mean	0.13	0.22	0.00	0.07	0.00	0.12	0.00	0.00	0.00	0.00	0.05	1.0
TabImpute	0.83	210.51	7.95	15.23	40.15	5.08	9.20	4.74	13.11	10.13	31.69	2.6
MissFI	34.20	131.10	23.87	22.45	29.93	16.16	18.37	20.30	22.57	32.28	35.12	3.8
GAIN	16.62	30.76	21.92	30.16	40.68	1518.47	48.31	107.95	21.38	32.06	186.83	4.5
PMAE	75.49	81.47	77.45	82.60	78.07	64.77	38.31	82.37	83.36	71.83	73.57	5.3
HoloClean	16.69	1235.24	56.47	57.32	229.22	102.37	113.35	126.14	90.19	89.42	211.64	6.4
DDGANI	151.89	581.10	271.28	105.39	366.07	364.33	71.47	55.61	270.15	213.38	245.07	8.0
Graf	91.05	91.73	76.61	89.27	1074.02	120.08	103.34	511.08	506.41	232.89	289.65	8.2
AimNet	17.35	2000.46	121.35	66.02	3697.12	255.32	238.12	265.35	240.45	120.23	702.18	8.7
Renuver	106.53	1994.45	560.62	1269.84	202.82	131.61	206.53	270.46	1271.87	761.85	677.66	10.1
TabDiff	300.28	726.46	611.55	780.35	678.54	535.20	559.18	456.37	460.21	480.04	558.82	10.3
DiffPuter	614.54	852.25	725.12	847.57	424.94	868.55	751.90	194.62	609.55	771.04	666.01	11.0
UnIMP	25.40	135.12	6779.90	16757.74	29049.07	10904.45	12863.53	14659.52	16025.11	17426.52	12462.64	12.4
TabCSDI	342.84	1967.08	1121.93	1207.71	2280.42	2114.69	1226.62	2354.15	1821.75	897.73	1533.49	12.7

TABLE XII: CPU Utilization Comparison
(Avg Rank is computed by ranking methods per dataset and averaging over all datasets)

(a) Average CPU Utilization (in %)

Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	AVG	Avg Rank ↓
PMAE	2.12	2.84	2.58	2.60	3.50	2.69	2.58	2.61	2.60	2.60	2.67	1.7
TabImpute	3.14	2.78	2.65	3.14	3.25	2.73	2.78	3.42	2.79	2.62	2.93	3.3
Mean	1.10	3.23	3.10	3.00	3.10	3.50	8.50	2.60	3.40	2.70	3.42	4.4
Renuver	3.28	53.91	2.54	3.48	3.09	3.22	3.09	3.13	3.01	2.61	8.14	4.5
DDGANI	3.11	3.12	3.14	3.13	3.59	3.57	3.15	5.01	3.16	3.18	3.42	5.2
TabCSDI	2.43	3.49	3.68	3.22	3.72	3.82	4.33	4.02	4.00	3.91	3.66	6.6
HoloClean	6.26	45.86	3.01	2.96	5.05	3.38	3.26	3.20	3.24	27.20	10.34	6.7
MissFI	2.89	17.58	4.31	5.02	6.14	5.59	3.79	3.19	7.83	6.90	6.32	8.0
TabDiff	3.71	8.40	8.28	8.34	5.83	2.87	5.88	13.16	9.56	8.40	7.44	8.9
AimNet	7.50	48.00	4.00	4.10	6.00	3.80	3.50	3.60	3.90	30.00	11.44	9.1
Graf	4.90	8.57	9.42	9.57	9.53	9.26	7.32	5.98	4.84	5.75	7.51	10.1
UnIMP	6.47	24.80	10.79	7.81	6.85	8.30	7.01	9.85	17.37	9.15	10.84	11.1
GAIN	5.37	50.12	5.21	52.47	50.68	50.44	52.35	52.37	5.13	52.49	37.66	12.7
DiffPuter	30.77	20.62	22.41	22.07	20.11	20.86	22.36	19.60	21.08	22.90	22.28	12.7

(b) Peak CPU Utilization (in %)

Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	MAX	Avg Rank ↓
Mean	2.20	3.60	3.10	3.00	3.10	3.50	8.50	2.60	3.40	2.70	8.50	1.9
TabImpute	8.20	14.50	4.30	11.50	20.70	16.30	5.80	15.80	6.20	3.80	20.70	3.3
PMAE	13.30	19.50	3.30	3.30	16.80	6.70	3.40	6.20	3.40	3.40	19.50	3.6
Renuver	17.80	90.10	6.70	6.90	12.50	25.20	19.60	20.20	14.60	19.30	90.10	5.0
MissFI	17.10	73.60	15.50	20.10	26.00	17.70	13.00	16.60	23.30	18.20	73.60	5.6
Graf	30.90	59.90	29.80	27.20	48.10	57.50	23.70	37.40	20.90	29.30	59.90	6.7
UnIMP	23.40	49.50	35.40	58.50	45.30	53.60	44.40	36.30	46.10	45.60	58.50	7.0
HoloClean	22.70	90.30	15.30	16.30	40.10	33.20	17.50	26.80	19.60	44.20	90.30	7.4
GAIN	50.00	53.80	32.40	54.00	66.20	65.90	78.00	54.60	12.30	53.90	78.00	7.6
TabDiff	18.80	58.10	22.70	28.50	68.40	66.30	18.30	70.00	60.00	21.40	70.00	7.8
AimNet	28.20	95.00	20.30	20.20	45.00	38.10	22.00	30.20	25.60	50.80	95.00	9.3
DDGANI	17.60	51.40	17.80	16.90	61.20	48.30	17.80	64.70	50.20	17.40	64.70	9.6
TabCSDI	22.60	60.00	25.20	22.00	80.00	18.20	20.00	25.10	26.70	25.20	80.00	9.8
DiffPuter	82.00	78.10	78.20	73.80	75.80	84.00	71.20	73.80	79.30	73.80	84.00	12.0

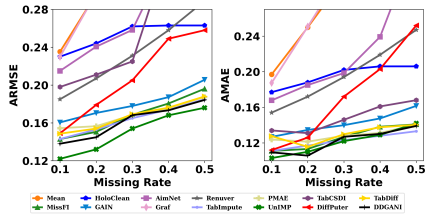
TABLE XIII: GPU Utilization Comparison
(Avg Rank is computed by ranking methods per dataset and averaging over all datasets)

(a) Average GPU Utilization (in %)

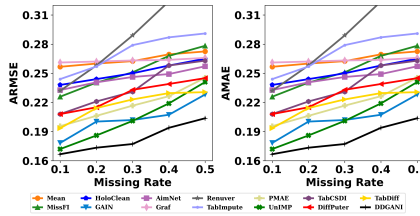
Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	AVG	Avg Rank ↓
GAIN	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Mean	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Renuver	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
MissFI	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
HoloClean	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Graf	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
AimNet	0.00	0.87	0.42	0.41	0.93	0.57	0.44	0.53	0.46	0.89	0.55	6.5
DiffPuter	0.69	1.45	0.61	0.57	1.05	1.00	0.62	18.22	0.74	0.57	2.55	8.2
DDGANI	2.02	2.53	2.07	2.22	1.34	1.91	1.37	0.94	1.85	2.00	1.83	9.3
PMAE	1.73	2.75	1.83	1.84	2.16	1.85	1.96	1.97	1.98	2.34	2.04	9.5
TabCSDI	3.24	7.49	2.47	2.30	3.04	2.98	2.31	2.65	2.37	2.69	3.15	11.1
TabDiff	22.68	22.15	52.40	51.38	19.87	3.18	20.09	27.68	37.32	53.97	31.07	12.7
TabImpute	38.35	29.28	30.45	49.92	53.59	26.18	29.45	19.76	30.38	30.56	33.79	12.7
UnIMP	2.92	5.55	94.94	98.34	98.10	97.47	98.27	96.77	97.27	98.69	78.83	13.4

(b) Peak GPU Utilization (in %)

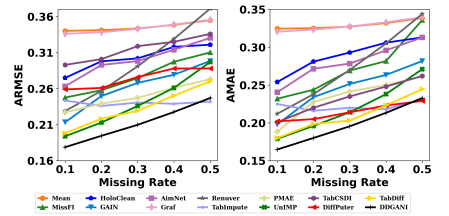
Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	MAX	Avg Rank ↓
GAIN	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Mean	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Renuver	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
MissFI	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
HoloClean	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
Graf	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.0
AimNet	0.00	4.00	3.00	3.00	5.00	5.30	3.10	4.20	3.20	4.00	5.30	5.8
PMAE	7.00	10.00	7.00	7.00	7.00	8.00	6.00	8.00	8.00	8.00	10.00	6.2
DDGANI	8.00	10.00	8.00	9.00	7.00	13.00	6.00	5.00	7.00	7.00	13.00	6.6
TabCSDI	10.00	25.00	13.00	13.00	18.00	12.00	12.00	13.00	11.00	11.00	25.00	7.5
DiffPuter	21.00	59.00	29.00	31.00	43.00	36.00	31.00	74.00	35.00	26.00	74.00	9.1
UnIMP	11.00	28.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	11.6
TabDiff	29.00	76.00	100.00	100.00	78.00	15.00	85.00	100.00	100.00	100.00	100.00	11.8
TabImpute	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	13.0



(a) Wine

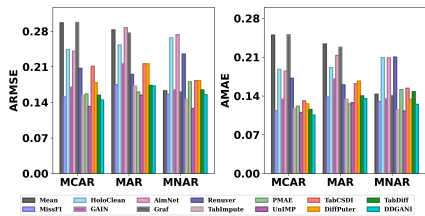


(b) Flare

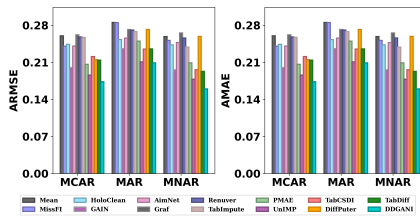


(c) Adult

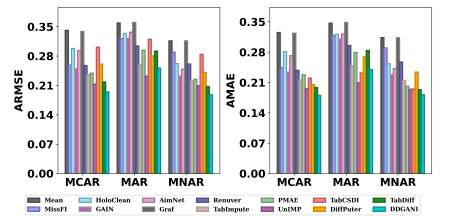
Fig. 6: Comparison of All Methods under Different Missing Rates



(a) Wine



(b) Flare



(c) Adult

Fig. 7: Comparison of All Methods under Different Missing Types

TABLE XIV: GPU Memory Consumption Comparison
(Avg Rank is computed by ranking methods per dataset and averaging over all datasets)

(a) Average GPU Memory Usage (in GB)

Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	AVG	Avg Rank ↓
GAIN	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	1.0
MissFI	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	1.0
Renuver	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	1.0
HoloClean	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	0.34	1.0
Graf	0.40	0.40	0.40	0.40	0.40	0.40	0.40	0.40	0.40	0.40	0.40	5.0
Mean	0.47	0.47	0.47	0.47	0.47	0.47	0.47	0.47	0.47	0.47	0.47	6.0
AimNet	0.48	0.50	0.50	0.50	0.53	0.54	0.51	0.51	0.51	0.52	0.51	7.0
PMAE	0.62	0.64	0.66	0.66	0.66	0.63	0.66	0.66	0.66	0.66	0.65	8.4
TabCSDI	1.26	1.79	1.73	1.61	1.69	1.60	1.73	1.73	1.73	1.73	1.66	9.6
DiffPuter	1.67	2.13	1.97	1.97	2.25	3.59	1.66	1.73	1.83	1.80	2.06	10.4
DDGANI	0.84	1.29	1.31	0.71	0.71	3.33	0.71	0.94	1.31	1.31	1.25	10.8
TabDiff	0.77	0.84	6.70	5.89	1.29	1.58	1.30	2.34	3.89	6.84	3.14	11.6
TabImpute	1.91	3.54	3.69	5.98	6.96	2.36	3.84	0.76	3.79	5.56	3.84	12.5
UnIMP	1.27	2.51	31.94	35.17	38.18	31.91	34.39	33.77	33.39	38.99	28.15	13.8

(b) Peak GPU Memory Usage (in GB)

Method	Wine	Spam	Tic	Flare	Adult	Food	Hosp.	Tax	Cont.	Aus.	MAX	Avg Rank ↓
GAIN	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	1.0
MissFI	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	1.0
HoloClean	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	1.0
Renuver	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	0.35	1.0
Graf	0.45	0.45	0.45	0.45	0.45	0.45	0.45	0.45	0.45	0.45	0.45	5.00
Mean	0.73	0.73	0.73	0.73	0.73	0.73	0.73	0.73	0.73	0.73	0.73	6.75
AimNet	0.66	0.69	0.70	0.69	0.75	0.75	0.70	0.70	0.71	0.73	0.75	6.25
PMAE	0.84	0.88	0.92	0.92	0.92	0.84	0.92	0.92	0.92	0.92	0.92	8.00
DiffPuter	1.95	2.33	2.33	2.33	2.52	2.14	2.33	4.78	2.33	2.33	4.78	11.00
TabCSDI	1.90	3.78	1.52	1.15	1.52	2.13	1.56	1.66	1.73	1.73	3.78	9.85
DDGANI	1.90	1.91	1.90	1.07	1.07	2.60	1.07	1.77	1.80	1.80	2.60	9.85
TabDiff	1.63	3.13	18.76	15.96	3.22	3.24	3.22	15.96	18.78	18.78	18.78	12.00
TabImpute	3.60	11.30	12.11	16.04	18.60	8.41	12.11	1.64	12.18	18.60	18.60	12.50
UnIMP	2.47	9.58	34.75	44.19	47.98	34.38	40.80	40.74	39.86	47.98	47.98	13.80

TABLE XV: Details of CFDs Refinement

Dataset	True CFDs	Potential CFDs	Refined CFDs	# Iterations
Food	Address→Zip, (_ _) Zip, DBA_Name→Address, (60622.0, _ _) Zip, DBA_Name→Address, (60636.0, _ _) Zip, DBA_Name→Address, (60619.0, _ _) Zip, DBA_Name→Address, (60601.0, _ _) Zip, DBA_Name→Address, (60616.0, _ _) Zip, DBA_Name→Address, (60652.0, _ _) Facility_Type, Address→DBA_Name, (school, _ _) DBA_Name, Inspection_Type→Address, (_ , license _) Facility_Type, DBA_Name→Address, (school, _ _) Facility_Type, DBA_Name→Address, (children's services facility, _ _) Facility_Type, Zip→Risk, (_ , 60607.0 _)	Address→Zip, (_ _) Zip, DBA_Name→Address, (60622.0, _ _) Zip, DBA_Name→Address, (60636.0, _ _) Zip, DBA_Name→Address, (60619.0, _ _) Zip, DBA_Name→Address, (60601.0, _ _) Zip, DBA_Name→Address, (60604.0, _ _) Zip, DBA_Name→Address, (60603.0, _ _) DBA_Name, Inspection_Type→Address, (_ , _ _) Facility_Type, DBA_Name→Address, (_ , _ _) Zip→Risk, (60607.0 _) Zip→Risk, (60616.0 _)	Address→Zip, (_ _) Zip, DBA_Name→Address, (60622.0, _ _) Zip, DBA_Name→Address, (60636.0, _ _) Zip, DBA_Name→Address, (60619.0, _ _) Zip, DBA_Name→Address, (60601.0, _ _) Zip, DBA_Name→Address, (60608.0, _ _) Zip, DBA_Name→Address, (60616.0, _ _) Zip, DBA_Name→Address, (60652.0, _ _) Facility_Type, Address→DBA_Name, (school, _ _) DBA_Name, Inspection_Type→Address, (_ , license _) Facility_Type, Zip→Risk, (_ , 60607.0 _) Facility_Type, DBA_Name→Address, (children's services facility, _ _) Facility_Type, DBA_Name→Address, (school, _ _)	8

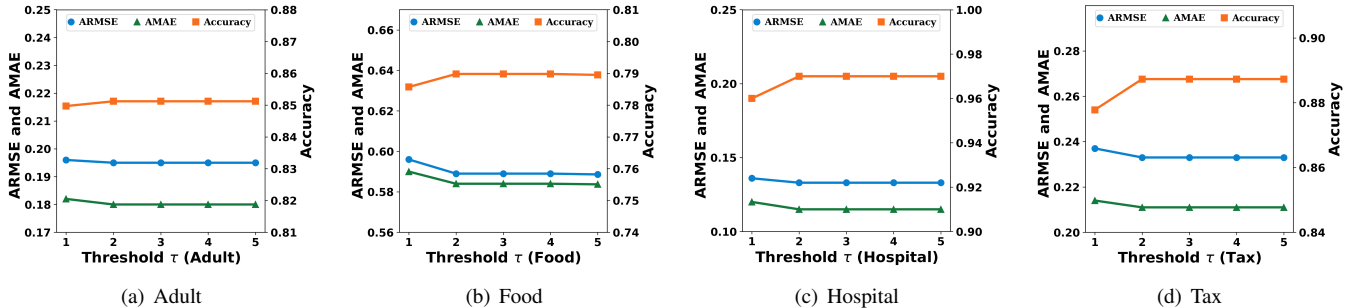


Fig. 8: Impact of Hyperparameter τ